

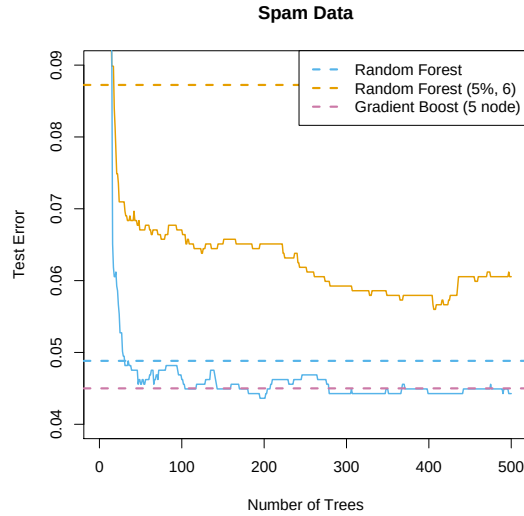


FIGURE 16.6. Application of the lasso post-processing (16.9) to the spam data. The horizontal blue line is the test error of a random forest fit to the spam data, using 1000 trees grown to maximum depth (with $m = 7$; see Algorithm 15.1). The jagged blue curve is the test error after post-processing the first 500 trees using the lasso, as a function of the number of trees with nonzero coefficients. The orange curve/line use a modified form of random forest, where a random draw of 5% of the data are used to grow each tree, and the trees are forced to be shallow (typically six terminal nodes). Here the post-processing offers much greater improvement over the random forest that generated the ensemble.

where they are needed, and are sufficiently different from each other for the post-processor to be effective.

Friedman and Popescu (2003) gain insights from numerical quadrature and importance sampling. They view the unknown function as an integral

$$f(x) = \int \beta(\gamma)b(x;\gamma)d\gamma, \quad (16.10)$$

where $\gamma \in \Gamma$ indexes the basis functions $b(x;\gamma)$. For example, if the basis functions are trees, then γ indexes the splitting variables, the split-points and the values in the terminal nodes. Numerical quadrature amounts to finding a set of M evaluation points $\gamma_m \in \Gamma$ and corresponding weights α_m so that $f_M(x) = \alpha_0 + \sum_{m=1}^M \alpha_m b(x;\gamma_m)$ approximates $f(x)$ well over the domain of x . Importance sampling amounts to sampling γ at random, but giving more weight to relevant regions of the space Γ . Friedman and Popescu (2003) suggest a measure of (lack of) relevance that uses the loss function (16.9):

$$Q(\gamma) = \min_{c_0, c_1} \sum_{i=1}^N L(y_i, c_0 + c_1 b(x_i; \gamma)), \quad (16.11)$$

evaluated on the training data.

If a single basis function were to be selected (e.g., a tree), it would be the global minimizer $\gamma^* = \arg \min_{\gamma \in \Gamma} Q(\gamma)$. Introducing randomness in the selection of γ would necessarily produce less optimal values with $Q(\gamma) \geq Q(\gamma^*)$. They propose a natural measure of the characteristic *width* σ of the sampling scheme $\mathcal{S}$,

$$\sigma = E_{\mathcal{S}}[Q(\gamma) - Q(\gamma^*)]. \quad (16.12)$$

- σ too narrow suggests too many of the $b(x; \gamma_m)$ look alike, and similar to $b(x; \gamma^*)$;
- σ too wide implies a large spread in the $b(x; \gamma_m)$, but possibly consisting of many irrelevant cases.

Friedman and Popescu (2003) use sub-sampling as a mechanism for introducing randomness, leading to their ensemble-generation algorithm 16.2.

Algorithm 16.2 *ISLE Ensemble Generation.*

1. $f_0(x) = \arg \min_c \sum_{i=1}^N L(y_i, c)$
 2. For $m = 1$ to M do
 - (a) $\gamma_m = \arg \min_{\gamma} \sum_{i \in S_m(\eta)} L(y_i, f_{m-1}(x_i) + b(x_i; \gamma))$
 - (b) $f_m(x) = f_{m-1}(x) + \nu b(x; \gamma_m)$
 3. $\mathcal{T}_{ISLE} = \{b(x; \gamma_1), b(x; \gamma_2), \dots, b(x; \gamma_M)\}$.
-

$S_m(\eta)$ refers to a subsample of $N \cdot \eta$ ($\eta \in (0, 1]$) of the training observations, typically *without* replacement. Their simulations suggest picking $\eta \leq \frac{1}{2}$, and for large N picking $\eta \sim 1/\sqrt{N}$. Reducing η increases the randomness, and hence the width σ . The parameter $\nu \in [0, 1]$ introduces *memory* into the randomization process; the larger ν , the more the procedure avoids $b(x; \gamma)$ similar to those found before. A number of familiar randomization schemes are special cases of Algorithm 16.2:

Bagging has $\eta = 1$, but samples with replacement, and has $\nu = 0$. Friedman and Hall (2007) argue that sampling without replacement with $\eta = 1/2$ is equivalent to sampling with replacement with $\eta = 1$, and the former is much more efficient.

Random forest sampling is similar, with more randomness introduced by the selection of the splitting variable. Reducing $\eta < 1/2$ in algorithm 16.2 has a similar effect to reducing m in random forests, but does not suffer from the potential biases discussed in Section 15.4.2.

Gradient boosting with shrinkage (10.41) uses $\eta = 1$, but typically does not produce sufficient width σ .

Stochastic gradient boosting (Friedman, 1999) follows the recipe exactly.

The authors recommend values $\nu = 0.1$ and $\eta \leq \frac{1}{2}$, and call their combined procedure (ensemble generation and post processing) *Importance sampled learning ensemble* (ISLE).

Figure 16.7 shows the performance of an ISLE on the spam data. It does

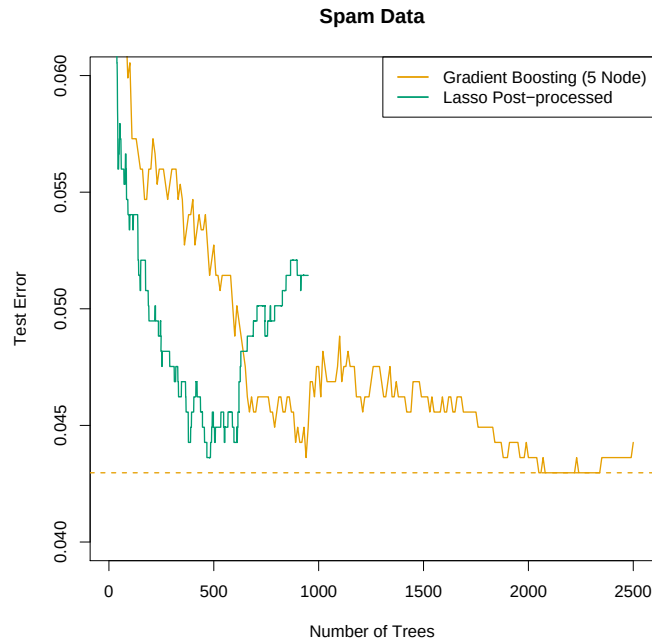


FIGURE 16.7. *Importance sampling learning ensemble (ISLE) fit to the spam data. Here we used $\eta = 1/2$, $\nu = 0.05$, and trees with five terminal nodes. The lasso post-processed ensemble does not improve the prediction error in this case, but it reduces the number of trees by a factor of five.*

not improve the predictive performance, but is able to produce a more parsimonious model. Note that in practice the post-processing includes the selection of the regularization parameter λ in (16.9), which would be

chosen by cross-validation. Here we simply demonstrate the effects of post-processing by showing the entire path on the test data.

Figure 16.8 shows various ISLEs on a regression example. The generating

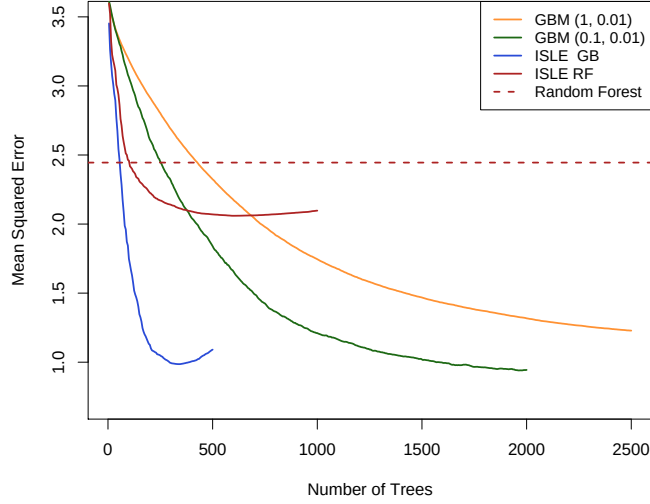


FIGURE 16.8. Demonstration of ensemble methods on a regression simulation example. The notation $GBM(0.1, 0.01)$ refers to a gradient boosted model, with parameters (η, ν) . We report mean-squared error from the true (known) function. Note that the sub-sampled GBM model (green) outperforms the full GBM model (orange). The lasso post-processed version achieves similar error. The random forest is outperformed by its post-processed version, but both fall short of the other models.

function is

$$f(X) = 10 \cdot \prod_{j=1}^5 e^{-2X_j^2} + \sum_{j=6}^{35} X_j, \quad (16.13)$$

where $X \sim U[0, 1]^{100}$ (the last 65 elements are noise variables). The response $Y = f(X) + \varepsilon$ where $\varepsilon \sim N(0, \sigma^2)$; we chose $\sigma = 1.3$ resulting in a signal-to-noise ratio of approximately 2. We used a training sample of size 1000, and estimated the mean squared error $E(\hat{f}(X) - f(X))^2$ by averaging over a test set of 500 samples. The sub-sampled GBM curve (light blue) is an instance of *stochastic gradient boosting* (Friedman, 1999) discussed in Section 10.12, and it outperforms gradient boosting on this example.

16.3.2 Rule Ensembles

Here we describe a modification of the tree-ensemble method that focuses on individual rules (Friedman and Popescu, 2003). We encountered rules in Section 9.3 in the discussion of the PRIM method. The idea is to enlarge an ensemble of trees by constructing a set of rules from each of the trees in the collection.

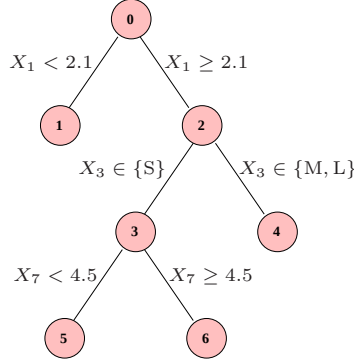


FIGURE 16.9. A typical tree in an ensemble, from which rules can be derived.

Figure 16.9 depicts a small tree, with numbered nodes. The following rules can be derived from this tree:

$$\begin{aligned}
 R_1(X) &= I(X_1 < 2.1) \\
 R_2(X) &= I(X_1 \geq 2.1) \\
 R_3(X) &= I(X_1 \geq 2.1) \cdot I(X_3 \in \{S\}) \\
 R_4(X) &= I(X_1 \geq 2.1) \cdot I(X_3 \in \{M, L\}) \\
 R_5(X) &= I(X_1 \geq 2.1) \cdot I(X_3 \in \{S\}) \cdot I(X_7 < 4.5) \\
 R_6(X) &= I(X_1 \geq 2.1) \cdot I(X_3 \in \{S\}) \cdot I(X_7 \geq 4.5)
 \end{aligned} \tag{16.14}$$

A linear expansion in rules 1, 4, 5 and 6 is equivalent to the tree itself (Exercise 16.3); hence (16.14) is an *over-complete* basis for the tree.

For each tree T_m in an ensemble $\mathcal{T}$, we can construct its mini-ensemble of rules $\mathcal{T}_{\text{RULE}}^m$, and then combine them all to form a larger ensemble

$$\mathcal{T}_{\text{RULE}} = \bigcup_{m=1}^M \mathcal{T}_{\text{RULE}}^m. \tag{16.15}$$

This is then treated like any other ensemble, and post-processed via the lasso or similar regularized procedure.

There are several advantages to this approach of deriving rules from the more complex trees:

- The space of models is enlarged, and can lead to improved performance.

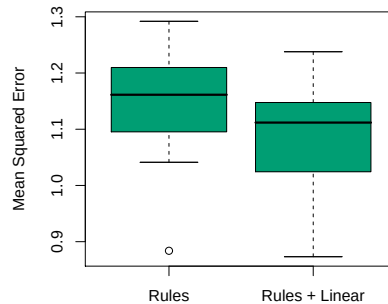


FIGURE 16.10. Mean squared error for rule ensembles, using 20 realizations of the simulation example (16.13).

- Rules are easier to interpret than trees, so there is the potential for a simplified model.
- It is often natural to augment $\mathcal{T}_{RULE}$ by including each variable X_j separately as well, thus allowing the ensemble to model linear functions well.

Friedman and Popescu (2008) demonstrate the power of this procedure on a number of illustrative examples, including the simulation example (16.13). Figure 16.10 shows boxplots of the mean-squared error from the true model for twenty realizations from this model. The models were all fit using the `Rulefit` software, available on the ESL homepage³, which runs in an automatic mode.

On the same training set as used in Figure 16.8, the rule based model achieved a mean-squared error of 1.06. Although slightly worse than the best achieved in that figure, the results are not comparable because cross-validation was used here to select the final model.

Bibliographic Notes

As noted in the introduction, many of the new methods in machine learning have been dubbed “ensemble” methods. These include neural networks boosting, bagging and random forests; Dietterich (2000a) gives a survey of tree-based ensemble methods. Neural networks (Chapter 11) are perhaps more deserving of the name, since they simultaneously learn the parameters

³ESL homepage: www-stat.stanford.edu/ElemStatLearn

of the hidden units (basis functions), along with how to combine them. Bishop (2006) discusses neural networks in some detail, along with the Bayesian perspective (MacKay, 1992; Neal, 1996). Support vector machines (Chapter 12) can also be regarded as an ensemble method; they perform L_2 regularized model fitting in high-dimensional feature spaces. Boosting and lasso exploit sparsity through L_1 regularization to overcome the high-dimensionality, while SVMs rely on the “kernel trick” characteristic of L_2 regularization.

C5.0 (Quinlan, 2004) is a commercial tree and rule generation package, with some goals in common with **Rulefit**.

There is a vast and varied literature often referred to as “combining classifiers” which abounds in ad-hoc schemes for mixing methods of different types to achieve better performance. For a principled approach, see Kittler et al. (1998).

Exercises

Ex. 16.1 Describe exactly how to generate the block correlated data used in the simulation in Section 16.2.3.

Ex. 16.2 Let $\alpha(t) \in \mathbb{R}^p$ be a piecewise-differentiable and continuous coefficient profile, with $\alpha(0) = 0$. The L_1 arc-length of α from time 0 to t is defined by

$$\Lambda(t) = \int_0^t |\dot{\alpha}(t)|_1 dt. \quad (16.16)$$

Show that $\Lambda(t) \geq |\alpha(t)|_1$, with equality iff $\alpha(t)$ is monotone.

Ex. 16.3 Show that fitting a linear regression model using rules 1, 4, 5 and 6 in equation (16.14) gives the same fit as the regression tree corresponding to this tree. Show the same is true for classification, if a logistic regression model is fit.

Ex. 16.4 Program and run the simulation study described in Figure 16.2.

17

Undirected Graphical Models

17.1 Introduction

A graph consists of a set of vertices (nodes), along with a set of edges joining some pairs of the vertices. In graphical models, each vertex represents a random variable, and the graph gives a visual way of understanding the joint distribution of the entire set of random variables. They can be useful for either unsupervised or supervised learning. In an *undirected graph*, the edges have no directional arrows. We restrict our discussion to undirected graphical models, also known as *Markov random fields* or *Markov networks*. In these graphs, the absence of an edge between two vertices has a special meaning: the corresponding random variables are conditionally independent, given the other variables.

Figure 17.1 shows an example of a graphical model for a flow-cytometry dataset with $p = 11$ proteins measured on $N = 7466$ cells, from Sachs et al. (2005). Each vertex in the graph corresponds to the real-valued expression level of a protein. The network structure was estimated assuming a multivariate Gaussian distribution, using the graphical lasso procedure discussed later in this chapter.

Sparse graphs have a relatively small number of edges, and are convenient for interpretation. They are useful in a variety of domains, including genomics and proteomics, where they provide rough models of cell pathways. Much work has been done in defining and understanding the structure of graphical models; see the Bibliographic Notes for references.

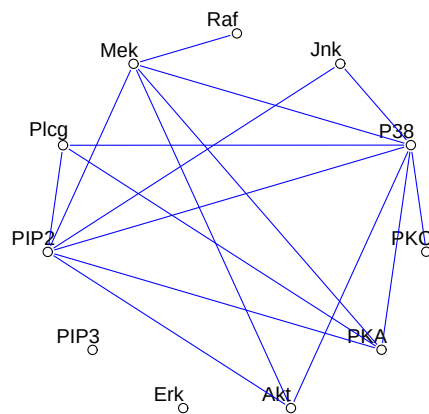


FIGURE 17.1. Example of a sparse undirected graph, estimated from a flow cytometry dataset, with $p = 11$ proteins measured on $N = 7466$ cells. The network structure was estimated using the graphical lasso procedure discussed in this chapter.

As we will see, the edges in a graph are parametrized by values or *potentials* that encode the strength of the conditional dependence between the random variables at the corresponding vertices. The main challenges in working with graphical models are model selection (choosing the structure of the graph), estimation of the edge parameters from data, and computation of marginal vertex probabilities and expectations, from their joint distribution. The last two tasks are sometimes called *learning* and *inference* in the computer science literature.

We do not attempt a comprehensive treatment of this interesting area. Instead, we introduce some basic concepts, and then discuss a few simple methods for estimation of the parameters and structure of undirected graphical models; methods that relate to the techniques already discussed in this book. The estimation approaches that we present for continuous and discrete-valued vertices are different, so we treat them separately. Sections 17.3.1 and 17.3.2 may be of particular interest, as they describe new, regression-based procedures for estimating graphical models.

There is a large and active literature on *directed graphical models* or *Bayesian networks*; these are graphical models in which the edges have directional arrows (but no directed cycles). Directed graphical models represent probability distributions that can be factored into products of conditional distributions, and have the potential for causal interpretations. We refer the reader to Wasserman (2004) for a brief overview of both undirected and directed graphs; the next section follows closely his Chapter 18.

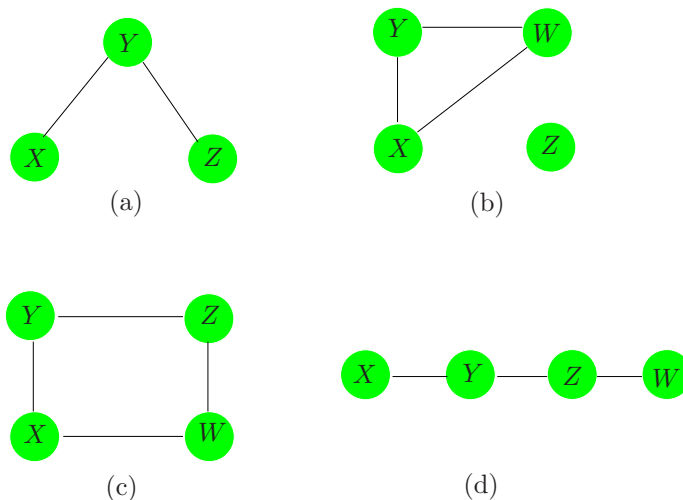


FIGURE 17.2. Examples of undirected graphical models or Markov networks. Each node or vertex represents a random variable, and the lack of an edge between two nodes indicates conditional independence. For example, in graph (a), X and Z are conditionally independent, given Y . In graph (b), Z is independent of each of X , Y , and W .

A longer list of useful references is given in the Bibliographic Notes on page 645.

17.2 Markov Graphs and Their Properties

In this section we discuss the basic properties of graphs as models for the joint distribution of a set of random variables. We defer discussion of (a) parametrization and estimation of the edge parameters from data, and (b) estimation of the topology of a graph, to later sections.

Figure 17.2 shows four examples of undirected graphs. A graph $\mathcal{G}$ consists of a pair (V, E) , where V is a set of vertices and E the set of edges (defined by pairs of vertices). Two vertices X and Y are called *adjacent* if there is an edge joining them; this is denoted by $X \sim Y$. A *path* $X_1, X_2, \dots, X_n$ is a set of vertices that are joined, that is $X_{i-1} \sim X_i$ for $i = 2, \dots, n$. A *complete graph* is a graph with every pair of vertices joined by an edge. A *subgraph* $U \in V$ is a subset of vertices together with their edges. For example, (X, Y, Z) in Figure 17.2(a) form a path but not a complete graph.

Suppose that we have a graph $\mathcal{G}$ whose vertex set V represents a set of random variables having joint distribution P . In a Markov graph $\mathcal{G}$, the absence of an edge implies that the corresponding random variables are conditionally independent given the variables at the other vertices. This is expressed with the following notation:

$$\text{No edge joining } X \text{ and } Y \iff X \perp Y | \text{rest} \quad (17.1)$$

where “rest” refers to all of the other vertices in the graph. For example in Figure 17.2(a) $X \perp Z | Y$. These are known as the *pairwise Markov independencies* of $\mathcal{G}$.

If A, B and C are subgraphs, then C is said to *separate* A and B if every path between A and B intersects a node in C . For example, Y separates X and Z in Figures 17.2(a) and (d), and Z separates Y and W in (d). In Figure 17.2(b) Z is not connected to X, Y, W so we say that the two sets are separated by the empty set. In Figure 17.2(c), $C = \{X, Z\}$ separates Y and W .

Separators have the nice property that they break the graph into conditionally independent pieces. Specifically, in a Markov graph $\mathcal{G}$ with subgraphs A, B and C ,

$$\text{if } C \text{ separates } A \text{ and } B \text{ then } A \perp B | C. \quad (17.2)$$

These are known as the *global Markov properties* of $\mathcal{G}$. It turns out that the pairwise and global Markov properties of a graph are equivalent (for graphs with positive distributions). That is, the set of graphs with associated probability distributions that satisfy the pairwise Markov independencies and global Markov assumptions are the same. This result is useful for inferring global independence relations from simple pairwise properties. For example in Figure 17.2(d) $X \perp Z | \{Y, W\}$ since it is a Markov graph and there is no link joining X and Z . But Y also separates X from Z and W and hence by the global Markov assumption we conclude that $X \perp Z | Y$ and $X \perp W | Y$. Similarly we have $Y \perp W | Z$.

The global Markov property allows us to decompose graphs into smaller more manageable pieces and thus leads to essential simplifications in computation and interpretation. For this purpose we separate the graph into cliques. A *clique* is a complete subgraph— a set of vertices that are all adjacent to one another; it is called *maximal* if it is a clique and no other vertices can be added to it and still yield a clique. The maximal cliques for the graphs of Figure 17.2 are

- (a) $\{X, Y\}, \{Y, Z\}$,
- (b) $\{X, Y, W\}, \{Z\}$,
- (c) $\{X, Y\}, \{Y, Z\}, \{Z, W\}, \{X, W\}$, and
- (d) $\{X, Y\}, \{Y, Z\}, \{Z, W\}$.

Although the following applies to both continuous and discrete distributions, much of the development has been for the latter. A probability density function f over a Markov graph $\mathcal{G}$ can be represented as

$$f(x) = \frac{1}{Z} \prod_{C \in \mathcal{C}} \psi_C(x_C) \quad (17.3)$$

where $\mathcal{C}$ is the set of maximal cliques, and the positive functions $\psi_C(\cdot)$ are called *clique potentials*. These are not in general density functions¹, but rather are affinities that capture the dependence in X_C by scoring certain instances x_C higher than others. The quantity

$$Z = \sum_{x \in \mathcal{X}} \prod_{C \in \mathcal{C}} \psi_C(x_C) \quad (17.4)$$

is the normalizing constant, also known as the *partition* function. Alternatively, the representation (17.3) implies a graph with independence properties defined by the cliques in the product. This result holds for Markov networks $\mathcal{G}$ with positive distributions, and is known as the *Hammersley-Clifford* theorem (Hammersley and Clifford, 1971; Clifford, 1990).

Many of the methods for estimation and computation on graphs first decompose the graph into its maximal cliques. Relevant quantities are computed in the individual cliques and then accumulated across the entire graph. A prominent example is the *join tree* or *junction tree* algorithm for computing marginal and low order probabilities from the joint distribution on a graph. Details can be found in Pearl (1986), Lauritzen and Spiegelhalter (1988), Pearl (1988), Shenoy and Shafer (1988), Jensen et al. (1990), or Koller and Friedman (2007).

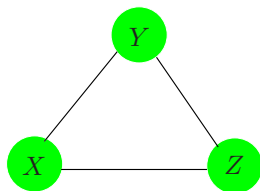


FIGURE 17.3. A complete graph does not uniquely specify the higher-order dependence structure in the joint distribution of the variables.

A graphical model does not always uniquely specify the higher-order dependence structure of a joint probability distribution. Consider the complete three-node graph in Figure 17.3. It could represent the dependence structure of either of the following distributions:

$$\begin{aligned} f^{(2)}(x, y, z) &= \frac{1}{Z} \psi(x, y) \psi(x, z) \psi(y, z); \\ f^{(3)}(x, y, z) &= \frac{1}{Z} \psi(x, y, z). \end{aligned} \quad (17.5)$$

The first specifies only second order dependence (and can be represented with fewer parameters). Graphical models for discrete data are a special

¹If the cliques are separated, then the potentials can be densities, but this is in general not the case.

case of *loglinear models for multiway contingency tables* (Bishop et al., 1975, e.g.); in that language $f^{(2)}$ is referred to as the “no second-order interaction” model.

For the remainder of this chapter we focus on *pairwise Markov graphs* (Koller and Friedman, 2007). Here there is a potential function for each edge (pair of variables as in $f^{(2)}$ above), and at most second-order interactions are represented. These are more parsimonious in terms of parameters, easier to work with, and give the minimal complexity implied by the graph structure. The models for both continuous and discrete data are functions of only the pairwise marginal distributions of the variables represented in the edge set.

17.3 Undirected Graphical Models for Continuous Variables

Here we consider Markov networks where all the variables are continuous. The Gaussian distribution is almost always used for such graphical models, because of its convenient analytical properties. We assume that the observations have a multivariate Gaussian distribution with mean μ and covariance matrix Σ . Since the Gaussian distribution represents at most second-order relationships, it automatically encodes a pairwise Markov graph. The graph in Figure 17.1 is an example of a Gaussian graphical model.

The Gaussian distribution has the property that all conditional distributions are also Gaussian. The inverse covariance matrix Σ^{-1} contains information about the *partial covariances* between the variables; that is, the covariances between pairs i and j , conditioned on all other variables. In particular, if the ij th component of $\Theta = \Sigma^{-1}$ is zero, then variables i and j are conditionally independent, given the other variables (Exercise 17.3).

It is instructive to examine the conditional distribution of one variable versus the rest, where the role of Θ is explicit. Suppose we partition $X = (Z, Y)$ where $Z = (X_1, \dots, X_{p-1})$ consists of the first $p - 1$ variables and $Y = X_p$ is the last. Then we have the conditional distribution of Y given Z (Mardia et al., 1979, e.g.)

$$Y|Z = z \sim N(\mu_Y + (z - \mu_Z)^T \Sigma_{ZZ}^{-1} \sigma_{ZY}, \sigma_{YY} - \sigma_{ZY}^T \Sigma_{ZZ}^{-1} \sigma_{ZY}), \quad (17.6)$$

where we have partitioned Σ as

$$\Sigma = \begin{pmatrix} \Sigma_{ZZ} & \sigma_{ZY} \\ \sigma_{ZY}^T & \sigma_{YY} \end{pmatrix}. \quad (17.7)$$

The conditional mean in (17.6) has exactly the same form as the population multiple linear regression of Y on Z , with regression coefficient $\beta = \Sigma_{ZZ}^{-1} \sigma_{ZY}$ [see (2.16) on page 19]. If we partition Θ in the same way, since $\Sigma\Theta = \mathbf{I}$ standard formulas for partitioned inverses give

$$\theta_{ZY} = -\theta_{YY} \cdot \Sigma_{ZZ}^{-1} \sigma_{ZY}, \quad (17.8)$$

where $1/\theta_{YY} = \sigma_{YY} - \sigma_{ZY}^T \Sigma_{ZZ}^{-1} \sigma_{ZY} > 0$. Hence

$$\begin{aligned} \beta &= \Sigma_{ZZ}^{-1} \sigma_{ZY} \\ &= -\theta_{ZY} / \theta_{YY}. \end{aligned} \quad (17.9)$$

We have learned two things here:

- The dependence of Y on Z in (17.6) is in the mean term alone. Here we see explicitly that zero elements in β and hence θ_{ZY} mean that the corresponding elements of Z are conditionally independent of Y , given the rest.
- We can learn about this dependence structure through multiple linear regression.

Thus Θ captures all the second-order information (both structural and quantitative) needed to describe the conditional distribution of each node given the rest, and is the so-called “natural” parameter for the Gaussian graphical model².

Another (different) kind of graphical model is the *covariance graph* or *relevance network*, in which vertices are connected by bidirectional edges if the covariance (rather than the partial covariance) between the corresponding variables is nonzero. These are popular in genomics, see especially Butte et al. (2000). The negative log-likelihood from these models is not convex, making the computations more challenging (Chaudhuri et al., 2007).

17.3.1 Estimation of the Parameters when the Graph Structure is Known

Given some realizations of X , we would like to estimate the parameters of an undirected graph that approximates their joint distribution. Suppose first that the graph is complete (fully connected). We assume that we have N multivariate normal realizations x_i , $i = 1, \dots, N$ with population mean μ and covariance Σ . Let

$$\mathbf{S} = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(x_i - \bar{x})^T \quad (17.10)$$

be the empirical covariance matrix, with $\bar{x}$ the sample mean vector. Ignoring constants, the log-likelihood of the data can be written as

²The distribution arising from a Gaussian graphical model is a Wishart distribution. This is a member of the exponential family, with canonical or “natural” parameter $\Theta = \Sigma^{-1}$. Indeed, the partially maximized log-likelihood (17.11) is (up to constants) the Wishart log-likelihood.

$$\ell(\Theta) = \log \det \Theta - \text{trace}(\mathbf{S}\Theta). \quad (17.11)$$

In (17.11) we have partially maximized with respect to the mean parameter μ . The quantity $-\ell(\Theta)$ is a convex function of Θ . It is easy to show that the maximum likelihood estimate of Σ is simply $\mathbf{S}$.

Now to make the graph more useful (especially in high-dimensional settings) let's assume that some of the edges are missing; for example, the edge between **PIP3** and **Erk** is one of several missing in Figure 17.1. As we have seen, for the Gaussian distribution this implies that the corresponding entries of $\Theta = \Sigma^{-1}$ are zero. Hence we now would like to maximize (17.11) under the constraints that some pre-defined subset of the parameters are zero. This is an equality-constrained convex optimization problem, and a number of methods have been proposed for solving it, in particular the iterative proportional fitting procedure (Speed and Kiiveri, 1986). This and other methods are summarized for example in Whittaker (1990) and Lauritzen (1996). These methods exploit the simplifications that arise from decomposing the graph into its maximal cliques, as described in the previous section. Here we outline a simple alternate approach, that exploits the sparsity in a different way. The fruits of this approach will become apparent later when we discuss the problem of estimation of the graph structure.

The idea is based on linear regression, as inspired by (17.6) and (17.9). In particular, suppose that we want to estimate the edge parameters θ_{ij} for the vertices that are joined to a given vertex i , restricting those that are not joined to be zero. Then it would seem that the linear regression of the node i values on the other relevant vertices might provide a reasonable estimate. But this ignores the dependence structure among the predictors in this regression. It turns out that if instead we use our current (model-based) estimate of the cross-product matrix of the predictors when we perform our regressions, this gives the correct solutions and solves the constrained maximum-likelihood problem exactly. We now give details.

To constrain the log-likelihood (17.11), we add Lagrange constants for all missing edges

$$\ell_C(\Theta) = \log \det \Theta - \text{trace}(\mathbf{S}\Theta) - \sum_{(j,k) \notin E} \gamma_{jk} \theta_{jk}. \quad (17.12)$$

The gradient equation for maximizing (17.12) can be written as

$$\Theta^{-1} - \mathbf{S} - \mathbf{\Gamma} = \mathbf{0}, \quad (17.13)$$

using the fact that the derivative of $\log \det \Theta$ equals Θ^{-1} (Boyd and Vandenberghe, 2004, for example, page 641). $\mathbf{\Gamma}$ is a matrix of Lagrange parameters with nonzero values for all pairs with edges absent.

We will show how we can use regression to solve for Θ and its inverse $\mathbf{W} = \Theta^{-1}$ one row and column at a time. For simplicity let's focus on the last row and column. Then the upper right block of equation (17.13) can be written as

$$w_{12} - s_{12} - \gamma_{12} = 0. \quad (17.14)$$

Here we have partitioned the matrices into two parts as in (17.7): part 1 being the first $p-1$ rows and columns, and part 2 the p th row and column. With $\mathbf{W}$ and its inverse Θ partitioned in a similar fashion, we have

$$\begin{pmatrix} \mathbf{W}_{11} & w_{12} \\ w_{12}^T & w_{22} \end{pmatrix} \begin{pmatrix} \Theta_{11} & \theta_{12} \\ \theta_{12}^T & \theta_{22} \end{pmatrix} = \begin{pmatrix} \mathbf{I} & 0 \\ 0^T & 1 \end{pmatrix}. \quad (17.15)$$

This implies

$$w_{12} = -\mathbf{W}_{11}\theta_{12}/\theta_{22} \quad (17.16)$$

$$= \mathbf{W}_{11}\beta \quad (17.17)$$

where $\beta = -\theta_{12}/\theta_{22}$ as in (17.9). Now substituting (17.17) into (17.14) gives

$$\mathbf{W}_{11}\beta - s_{12} - \gamma_{12} = 0. \quad (17.18)$$

These can be interpreted as the $p-1$ estimating equations for the constrained regression of X_p on the other predictors, except that the observed mean cross-products matrix $\mathbf{S}_{11}$ is replaced by $\mathbf{W}_{11}$, the current estimated covariance matrix from the model.

Now we can solve (17.18) by simple subset regression. Suppose there are $p-q$ nonzero elements in γ_{12} —i.e., $p-q$ edges constrained to be zero. These $p-q$ rows carry no information and can be removed. Furthermore we can reduce β to β^* by removing its $p-q$ zero elements, yielding the reduced $q \times q$ system of equations

$$\mathbf{W}_{11}^*\beta^* - s_{12}^* = 0, \quad (17.19)$$

with solution $\hat{\beta}^* = \mathbf{W}_{11}^{*-1}s_{12}^*$. This is padded with $p-q$ zeros to give $\hat{\beta}$.

Although it appears from (17.16) that we only recover the elements θ_{12} up to a scale factor $1/\theta_{22}$, it is easy to show that

$$\frac{1}{\theta_{22}} = w_{22} - w_{12}^T\beta \quad (17.20)$$

(using partitioned inverse formulas). Also $w_{22} = s_{22}$, since the diagonal of Γ in (17.13) is zero.

This leads to the simple iterative procedure given in Algorithm 17.1 for estimating both $\hat{\mathbf{W}}$ and its inverse $\hat{\Theta}$, subject to the constraints of the missing edges.

Note that this algorithm makes conceptual sense. The graph estimation problem is not p separate regression problems, but rather p coupled problems. The use of the common $\mathbf{W}$ in step (b), in place of the observed cross-products matrix, couples the problems together in the appropriate fashion. Surprisingly, we were not able to find this procedure in the literature. However it is related to the covariance selection procedures of

Algorithm 17.1 *A Modified Regression Algorithm for Estimation of an Undirected Gaussian Graphical Model with Known Structure.*

1. Initialize $\mathbf{W} = \mathbf{S}$.
 2. Repeat for $j = 1, 2, \dots, p, 1, \dots$ until convergence:
 - (a) Partition the matrix $\mathbf{W}$ into part 1: all but the j th row and column, and part 2: the j th row and column.
 - (b) Solve $\mathbf{W}_{11}^* \beta^* - s_{12}^* = 0$ for the unconstrained edge parameters β^* , using the reduced system of equations as in (17.19). Obtain $\hat{\beta}$ by padding $\hat{\beta}^*$ with zeros in the appropriate positions.
 - (c) Update $w_{12} = \mathbf{W}_{11} \hat{\beta}$
 3. In the final cycle (for each j) solve for $\hat{\theta}_{12} = -\hat{\beta} \cdot \hat{\theta}_{22}$, with $1/\hat{\theta}_{22} = s_{22} - w_{12}^T \hat{\beta}$.
-

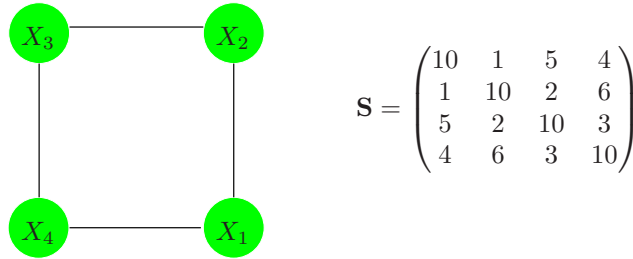


FIGURE 17.4. A simple graph for illustration, along with the empirical covariance matrix.

Dempster (1972), and is similar in flavor to the iterative conditional fitting procedure for covariance graphs, proposed by Chaudhuri et al. (2007).

Here is a little example, borrowed from Whittaker (1990). Suppose that our model is as depicted in Figure 17.4, along with its empirical covariance matrix $\mathbf{S}$. We apply algorithm (17.1) to this problem; for example, in the modified regression for variable 1 in step (b), variable 3 is left out. The procedure quickly converged to the solutions:

$$\hat{\Sigma} = \begin{pmatrix} 10.00 & 1.00 & \mathbf{1.31} & 4.00 \\ 1.00 & 10.00 & 2.00 & \mathbf{0.87} \\ \mathbf{1.31} & 2.00 & 10.00 & 3.00 \\ 4.00 & \mathbf{0.87} & 3.00 & 10.00 \end{pmatrix}, \quad \hat{\Sigma}^{-1} = \begin{pmatrix} 0.12 & -0.01 & \mathbf{0.00} & -0.05 \\ -0.01 & 0.11 & -0.02 & \mathbf{0.00} \\ \mathbf{0.00} & -0.02 & 0.11 & -0.03 \\ -0.05 & \mathbf{0.00} & -0.03 & 0.13 \end{pmatrix}.$$

Note the zeroes in $\hat{\Sigma}^{-1}$, corresponding to the missing edges (1,3) and (2,4). Note also that the corresponding elements in $\hat{\Sigma}$ are the only elements different from $\mathbf{S}$. The estimation of $\hat{\Sigma}$ is an example of what is sometimes called the positive definite “completion” of $\mathbf{S}$.

17.3.2 Estimation of the Graph Structure

In most cases we do not know which edges to omit from our graph, and so would like to try to discover this from the data itself. In recent years a number of authors have proposed the use of L_1 (lasso) regularization for this purpose.

Meinshausen and Bühlmann (2006) take a simple approach to the problem: rather than trying to fully estimate Σ or $\Theta = \Sigma^{-1}$, they only estimate which components of θ_{ij} are nonzero. To do this, they fit a lasso regression using each variable as the response and the others as predictors. The component θ_{ij} is then estimated to be nonzero if either the estimated coefficient of variable i on j is nonzero, OR the estimated coefficient of variable j on i is nonzero (alternatively they use an AND rule). They show that asymptotically this procedure consistently estimates the set of nonzero elements of Θ .

We can take a more systematic approach with the lasso penalty, following the development of the previous section. Consider maximizing the penalized log-likelihood

$$\log \det \Theta - \text{trace}(\mathbf{S}\Theta) - \lambda \|\Theta\|_1, \quad (17.21)$$

where $\|\Theta\|_1$ is the L_1 norm—the sum of the absolute values of the elements of Σ^{-1} , and we have ignored constants. The negative of this penalized likelihood is a convex function of Θ .

It turns out that one can adapt the lasso to give the exact maximizer of the penalized log-likelihood. In particular, we simply replace the modified regression step (b) in Algorithm 17.1 by a modified lasso step. Here are the details.

The analog of the gradient equation (17.13) is now

$$\Theta^{-1} - \mathbf{S} - \lambda \cdot \text{Sign}(\Theta) = \mathbf{0}. \quad (17.22)$$

Here we use *sub-gradient* notation, with $\text{Sign}(\theta_{jk}) = \text{sign}(\theta_{jk})$ if $\theta_{jk} \neq 0$, else $\text{Sign}(\theta_{jk}) \in [-1, 1]$ if $\theta_{jk} = 0$. Continuing the development in the previous section, we reach the analog of (17.18)

$$\mathbf{W}_{11}\beta - s_{12} + \lambda \cdot \text{Sign}(\beta) = 0 \quad (17.23)$$

(recall that β and θ_{12} have opposite signs). We will now see that this system is exactly equivalent to the estimating equations for a lasso regression.

Consider the usual regression setup with outcome variables $\mathbf{y}$ and predictor matrix $\mathbf{Z}$. There the lasso minimizes

$$\frac{1}{2}(\mathbf{y} - \mathbf{Z}\beta)^T(\mathbf{y} - \mathbf{Z}\beta) + \lambda \cdot \|\beta\|_1 \quad (17.24)$$

[see (3.52) on page 68; here we have added a factor $\frac{1}{2}$ for convenience]. The gradient of this expression is

Algorithm 17.2 *Graphical Lasso.*

-
1. Initialize $\mathbf{W} = \mathbf{S} + \lambda \mathbf{I}$. The diagonal of $\mathbf{W}$ remains unchanged in what follows.
 2. Repeat for $j = 1, 2, \dots, p, 1, 2, \dots, p, \dots$ until convergence:
 - (a) Partition the matrix $\mathbf{W}$ into part 1: all but the j th row and column, and part 2: the j th row and column.
 - (b) Solve the estimating equations $\mathbf{W}_{11}\beta - s_{12} + \lambda \cdot \text{Sign}(\beta) = 0$ using the cyclical coordinate-descent algorithm (17.26) for the modified lasso.
 - (c) Update $w_{12} = \mathbf{W}_{11}\hat{\beta}$
 3. In the final cycle (for each j) solve for $\hat{\theta}_{12} = -\hat{\beta} \cdot \hat{\theta}_{22}$, with $1/\hat{\theta}_{22} = w_{22} - w_{12}^T \hat{\beta}$.
-

$$\mathbf{Z}^T \mathbf{Z} \beta - \mathbf{Z}^T \mathbf{y} + \lambda \cdot \text{Sign}(\beta) = 0 \quad (17.25)$$

So up to a factor $1/N$, $\mathbf{Z}^T \mathbf{y}$ is the analog of s_{12} , and we replace $\mathbf{Z}^T \mathbf{Z}$ by $\mathbf{W}_{11}$, the estimated cross-product matrix from our current model.

The resulting procedure is called the *graphical lasso*, proposed by Friedman et al. (2008b) building on the work of Banerjee et al. (2008). It is summarized in Algorithm 17.2.

Friedman et al. (2008b) use the pathwise coordinate descent method (Section 3.8.6) to solve the modified lasso problem at each stage. Here are the details of pathwise coordinate descent for the graphical lasso algorithm. Letting $\mathbf{V} = \mathbf{W}_{11}$, the update has the form

$$\hat{\beta}_j \leftarrow S\left(s_{12j} - \sum_{k \neq j} V_{kj} \hat{\beta}_k, \lambda\right) / V_{jj} \quad (17.26)$$

for $j = 1, 2, \dots, p - 1, 1, 2, \dots, p - 1, \dots$, where S is the soft-threshold operator:

$$S(x, t) = \text{sign}(x)(|x| - t)_+. \quad (17.27)$$

The procedure cycles through the predictors until convergence.

It is easy to show that the diagonal elements w_{jj} of the solution matrix $\mathbf{W}$ are simply $s_{jj} + \lambda$, and these are fixed in step 1 of Algorithm 17.2³.

The graphical lasso algorithm is extremely fast, and can solve a moderately sparse problem with 1000 nodes in less than a minute. It is easy to modify the algorithm to have edge-specific penalty parameters λ_{jk} ; since

³An alternative formulation of the problem (17.21) can be posed, where we don't penalize the diagonal of Θ . Then the diagonal elements w_{jj} of the solution matrix are s_{jj} , and the rest of the algorithm is unchanged.

$\lambda_{jk} = \infty$ will force $\hat{\theta}_{jk}$ to be zero, this algorithm subsumes Algorithm 17.1. By casting the sparse inverse-covariance problem as a series of regressions, one can also quickly compute and examine the solution paths as a function of the penalty parameter λ . More details can be found in Friedman et al. (2008b).

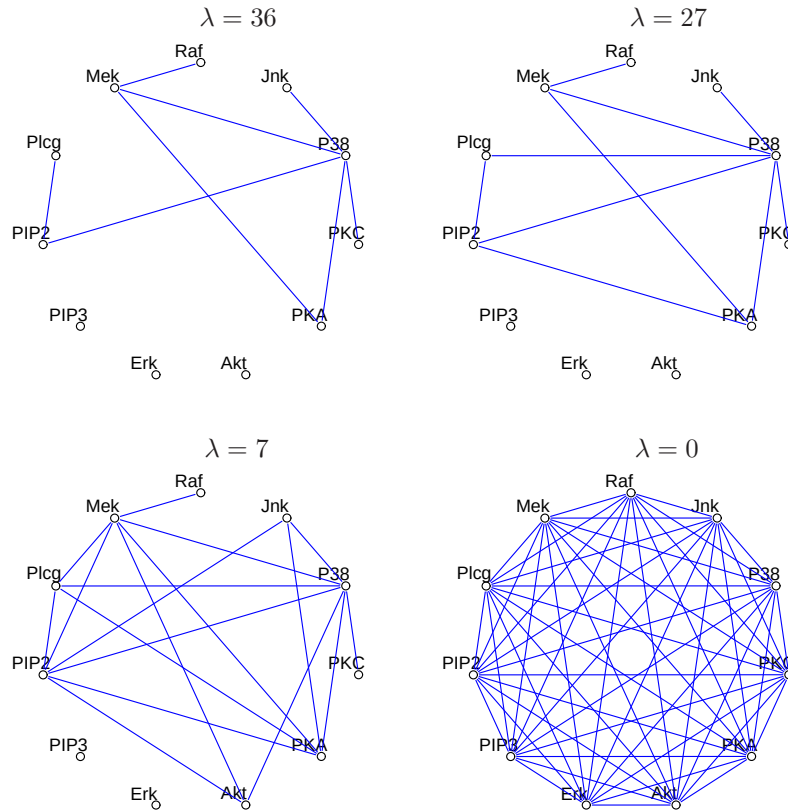


FIGURE 17.5. Four different graphical-lasso solutions for the flow-cytometry data.

Figure 17.1 shows the result of applying the graphical lasso to the flow-cytometry dataset. Here the lasso penalty parameter λ was set at 14. In practice it is informative to examine the different sets of graphs that are obtained as λ is varied. Figure 17.5 shows four different solutions. The graph becomes more sparse as the penalty parameter is increased.

Finally note that the values at some of the nodes in a graphical model can be unobserved; that is, missing or hidden. If only some values are missing at a node, the EM algorithm can be used to impute the missing values

(Exercise 17.9). However, sometimes the entire node is hidden or *latent*. In the Gaussian model, if a node has all missing values, due to linearity one can simply average over the missing nodes to yield another Gaussian model over the observed nodes. Hence the inclusion of hidden nodes does not enrich the resulting model for the observed nodes; in fact, it imposes additional structure on its covariance matrix. However in the discrete model (described next) the inherent nonlinearities make hidden units a powerful way of expanding the model.

17.4 Undirected Graphical Models for Discrete Variables

Undirected Markov networks with all discrete variables are popular, and in particular pairwise Markov networks with binary variables being the most common. They are sometimes called *Ising models* in the statistical mechanics literature, and *Boltzmann machines* in the machine learning literature, where the vertices are referred to as “nodes” or “units” and are binary-valued.

In addition, the values at each node can be observed (“visible”) or unobserved (“hidden”). The nodes are often organized in layers, similar to a neural network. Boltzmann machines are useful both for unsupervised and supervised learning, especially for structured input data such as images, but have been hampered by computational difficulties. Figure 17.6 shows a restricted Boltzmann machine (discussed later), in which some variables are hidden, and only some pairs of nodes are connected. We first consider the simpler case in which all p nodes are visible with edge pairs (j, k) enumerated in E .

Denoting the binary valued variable at node j by X_j , the Ising model for their joint probabilities is given by

$$p(X, \Theta) = \exp \left[\sum_{(j,k) \in E} \theta_{jk} X_j X_k - \Phi(\Theta) \right] \text{ for } X \in \mathcal{X}, \quad (17.28)$$

with $\mathcal{X} = \{0, 1\}^p$. As with the Gaussian model of the previous section, only pairwise interactions are modeled. The Ising model was developed in statistical mechanics, and is now used more generally to model the joint effects of pairwise interactions. $\Phi(\Theta)$ is the log of the partition function, and is defined by

$$\Phi(\Theta) = \log \sum_{x \in \mathcal{X}} \left[\exp \left(\sum_{(j,k) \in E} \theta_{jk} x_j x_k \right) \right]. \quad (17.29)$$

The partition function ensures that the probabilities add to one over the sample space. The terms $\theta_{jk} X_j X_k$ represent a particular parametrization

of the (log) potential functions (17.5), and for technical reasons requires a *constant* node $X_0 \equiv 1$ to be included (Exercise 17.10), with “edges” to all the other nodes. In the statistics literature, this model is equivalent to a first-order-interaction Poisson log-linear model for multiway tables of counts (Bishop et al., 1975; McCullagh and Nelder, 1989; Agresti, 2002).

The Ising model implies a logistic form for each node conditional on the others (exercise 17.11):

$$\Pr(X_j = 1 | X_{-j} = x_{-j}) = \frac{1}{1 + \exp(-\theta_{j0} - \sum_{(j,k) \in E} \theta_{jk} x_k)}, \quad (17.30)$$

where X_{-j} denotes all of the nodes except j . Hence the parameter θ_{jk} measures the dependence of X_j on X_k , conditional on the other nodes.

17.4.1 Estimation of the Parameters when the Graph Structure is Known

Given some data from this model, how can we estimate the parameters? Suppose we have observations $x_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \in \{0, 1\}^p$, $i = 1, \dots, N$. The log-likelihood is

$$\begin{aligned} \ell(\Theta) &= \sum_{i=1}^N \log \Pr_{\Theta}(X_i = x_i) \\ &= \sum_{i=1}^N \left[\sum_{(j,k) \in E} \theta_{jk} x_{ij} x_{ik} - \Phi(\Theta) \right] \end{aligned} \quad (17.31)$$

The gradient of the log-likelihood is

$$\frac{\partial \ell(\Theta)}{\partial \theta_{jk}} = \sum_{i=1}^N x_{ij} x_{ik} - N \frac{\partial \Phi(\Theta)}{\partial \theta_{jk}} \quad (17.32)$$

and

$$\begin{aligned} \frac{\partial \Phi(\Theta)}{\partial \theta_{jk}} &= \sum_{x \in \mathcal{X}} x_j x_k \cdot p(x, \Theta) \\ &= \mathbb{E}_{\Theta}(X_j X_k) \end{aligned} \quad (17.33)$$

Setting the gradient to zero gives

$$\hat{\mathbb{E}}(X_j X_k) - \mathbb{E}_{\Theta}(X_j X_k) = 0 \quad (17.34)$$

where we have defined

$$\hat{E}(X_j X_k) = \frac{1}{N} \sum_{i=1}^N x_{ij} x_{ik}, \quad (17.35)$$

the expectation taken with respect to the empirical distribution of the data. Looking at (17.34), we see that the maximum likelihood estimates simply match the estimated inner products between the nodes to their observed inner products. This is a standard form for the score (gradient) equation for exponential family models, in which sufficient statistics are set equal to their expectations under the model.

To find the maximum likelihood estimates, we can use gradient search or Newton methods. However the computation of $E_{\Theta}(X_j X_k)$ involves enumeration of $p(X, \Theta)$ over 2^{p-2} of the $|\mathcal{X}| = 2^p$ possible values of X , and is not generally feasible for large p (e.g., larger than about 30). For smaller p , a number of standard statistical approaches are available:

Poisson log-linear modeling, where we treat the problem as a large regression problem (Exercise 17.12). The response vector $\mathbf{y}$ is the vector of 2^p counts in each of the cells of the multiway tabulation of the data⁴. The predictor matrix $\mathbf{Z}$ has 2^p rows and up to $1+p+p^2$ columns that characterize each of the cells, although this number depends on the sparsity of the graph. The computational cost is essentially that of a regression problem of this size, which is $O(p^4 2^p)$ and is manageable for $p < 20$. The Newton updates are typically computed by iteratively reweighted least squares, and the number of steps is usually in the single digits. See Agresti (2002) and McCullagh and Nelder (1989) for details. Standard software (such as the R package `glm`) can be used to fit this model.

Gradient descent requires at most $O(p^2 2^{p-2})$ computations to compute the gradient, but may require many more gradient steps than the second-order Newton methods. Nevertheless, it can handle slightly larger problems with $p \leq 30$. These computations can be reduced by exploiting the special clique structure in sparse graphs, using the junction-tree algorithm. Details are not given here.

Iterative proportional fitting (IPF) performs cyclical coordinate descent on the gradient equations (17.34). At each step a parameter is updated so that its gradient equation is exactly zero. This is done in a cyclical fashion until all the gradients are zero. One complete cycle costs the same as a gradient evaluation, but may be more efficient. Jiroušek and Přeučil (1995) implement an efficient version of IPF, using junction trees.

⁴Each of the cell counts is treated as an independent Poisson variable. We get the multinomial model corresponding to (17.28) by conditioning on the total count N (which is also Poisson under this framework).

When p is large (> 30) other approaches have been used to approximate the gradient.

- The mean field approximation (Peterson and Anderson, 1987) estimates $E_{\Theta}(X_j X_k)$ by $E_{\Theta}(X_j)E_{\Theta}(X_k)$, and replaces the input variables by their means, leading to a set of nonlinear equations for the parameters θ_{jk} .
- To obtain near-exact solutions, Gibbs sampling (Section 8.6) is used to approximate $E_{\Theta}(X_j X_k)$ by successively sampling from the estimated model probabilities $\Pr_{\Theta}(X_j | X_{-j})$ (see e.g. Ripley (1996)).

We have not discussed *decomposable models*, for which the maximum likelihood estimates can be found in closed form without any iteration whatsoever. These models arise, for example, in *trees*: special graphs with tree-structured topology. When computational tractability is a concern, trees represent a useful class of models and they sidestep the computational concerns raised in this section. For details, see for example Chapter 12 of Whittaker (1990).

17.4.2 Hidden Nodes



We can increase the complexity of a discrete Markov network by including latent or hidden nodes. Suppose that a subset of the variables $X_{\mathcal{H}}$ are unobserved or “hidden”, and the remainder $X_{\mathcal{V}}$ are observed or “visible.” Then the log-likelihood of the observed data is

$$\begin{aligned} \ell(\Theta) &= \sum_{i=1}^N \log[\Pr_{\Theta}(X_{\mathcal{V}} = x_{i\mathcal{V}})] \\ &= \sum_{i=1}^N \left[\log \sum_{x_{\mathcal{H}} \in \mathcal{X}_{\mathcal{H}}} \exp \sum_{(j,k) \in E} (\theta_{jk} x_{ij} x_{ik} - \Phi(\Theta)) \right]. \end{aligned} \quad (17.36)$$

The sum over $x_{\mathcal{H}}$ means that we are summing over all possible $\{0, 1\}$ values for the hidden units. The gradient works out to be

$$\frac{d\ell(\Theta)}{d\theta_{jk}} = \hat{E}_{\mathcal{V}} E_{\Theta}(X_j X_k | X_{\mathcal{V}}) - E_{\Theta}(X_j X_k) \quad (17.37)$$

The first term is an empirical average of $X_j X_k$ if both are visible; if one or both are hidden, they are first imputed given the visible data, and then averaged over the hidden variables. The second term is the unconditional expectation of $X_j X_k$.

The inner expectation in the first term can be evaluated using basic rules of conditional expectation and properties of Bernoulli random variables. In detail, for observation i

$$E_{\Theta}(X_j X_k | X_{\mathcal{V}} = x_{i\mathcal{V}}) = \begin{cases} x_{ij} x_{ik} & \text{if } j, k \in \mathcal{V} \\ x_{ij} \Pr_{\Theta}(X_k = 1 | X_{\mathcal{V}} = x_{i\mathcal{V}}) & \text{if } j \in \mathcal{V}, k \in \mathcal{H} \\ \Pr_{\Theta}(X_j = 1, X_k = 1 | X_{\mathcal{V}} = x_{i\mathcal{V}}) & \text{if } j, k \in \mathcal{H}. \end{cases} \quad (17.38)$$

Now two separate runs of Gibbs sampling are required; the first to estimate $E_{\Theta}(X_j X_k)$ by sampling from the model as above, and the second to estimate $E_{\Theta}(X_j X_k | X_{\mathcal{V}} = x_{i\mathcal{V}})$. In this latter run, the visible units are fixed (“clamped”) at their observed values and only the hidden variables are sampled. Gibbs sampling must be done for each observation in the training set, at each stage of the gradient search. As a result this procedure can be very slow, even for moderate-sized models. In Section 17.4.4 we consider further model restrictions to make these computations manageable.

17.4.3 Estimation of the Graph Structure

The use of a lasso penalty with binary pairwise Markov networks has been suggested by Lee et al. (2007) and Wainwright et al. (2007). The first authors investigate a conjugate gradient procedure for exact maximization of a penalized log-likelihood. The bottleneck is the computation of $E_{\Theta}(X_j X_k)$ in the gradient; exact computation via the junction tree algorithm is manageable for sparse graphs but becomes unwieldy for dense graphs.

The second authors propose an approximate solution, analogous to the Meinshausen and Bühlmann (2006) approach for the Gaussian graphical model. They fit an L_1 -penalized logistic regression model to each node as a function of the other nodes, and then symmetrize the edge parameter estimates in some fashion. For example if $\tilde{\theta}_{jk}$ is the estimate of the j - k edge parameter from the logistic model for outcome node j , the “min” symmetrization sets $\hat{\theta}_{jk}$ to either $\tilde{\theta}_{jk}$ or $\tilde{\theta}_{kj}$, whichever is smallest in absolute value. The “max” criterion is defined similarly. They show that under certain conditions either approximation estimates the nonzero edges correctly as the sample size goes to infinity. Hoefling and Tibshirani (2008) extend the graphical lasso to discrete Markov networks, obtaining a procedure which is somewhat faster than conjugate gradients, but still must deal with computation of $E_{\Theta}(X_j X_k)$. They also compare the exact and approximate solutions in an extensive simulation study and find the “min” or “max” approximations are only slightly less accurate than the exact procedure, both for estimating the nonzero edges and for estimating the actual values of the edge parameters, and are *much* faster. Furthermore, they can handle denser graphs because they never need to compute the quantities $E_{\Theta}(X_j X_k)$.

Finally, we point out a key difference between the Gaussian and binary models. In the Gaussian case, both Σ and its inverse will often be of interest, and the graphical lasso procedure delivers estimates for both of these quantities. However, the approximation of Meinshausen and Bühlmann (2006) for Gaussian graphical models, analogous to the Wainwright et al. (2007)

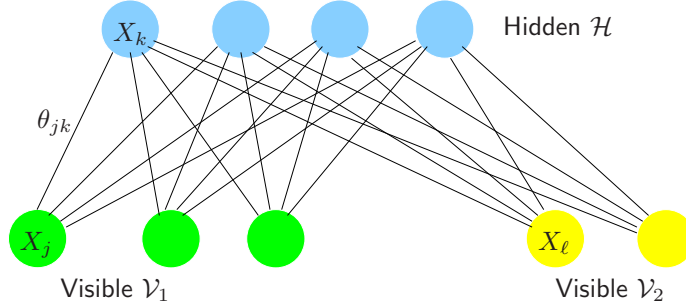


FIGURE 17.6. A restricted Boltzmann machine (RBM) in which there are no connections between nodes in the same layer. The visible units are subdivided to allow the RBM to model the joint density of feature $\mathcal{V}_1$ and their labels $\mathcal{V}_2$.

approximation for the binary case, only yields an estimate of Σ^{-1} . In contrast, in the Markov model for binary data, Θ is the object of interest, and its inverse is not of interest. The approximate method of Wainwright et al. (2007) estimates Θ efficiently and hence is an attractive solution for the binary problem.

17.4.4 Restricted Boltzmann Machines

In this section we consider a particular architecture for graphical models inspired by neural networks, where the units are organized in layers. A restricted Boltzmann machine (RBM) consists of one layer of visible units and one layer of hidden units with no connections within each layer. It is much simpler to compute the conditional expectations (as in (17.37) and (17.38)) if the connections between hidden units are removed⁵. Figure 17.6 shows an example; the visible layer is divided into input variables $\mathcal{V}_1$ and output variables $\mathcal{V}_2$, and there is a hidden layer $\mathcal{H}$. We denote such a network by

$$\mathcal{V}_1 \leftrightarrow \mathcal{H} \leftrightarrow \mathcal{V}_2. \quad (17.39)$$

For example, $\mathcal{V}_1$ could be the binary pixels of an image of a handwritten digit, and $\mathcal{V}_2$ could have 10 units, one for each of the observed class labels 0-9.

The restricted form of this model simplifies the Gibbs sampling for estimating the expectations in (17.37), since the variables in each layer are independent of one another, given the variables in the other layers. Hence they can be sampled together, using the conditional probabilities given by expression (17.30).

The resulting model is less general than a Boltzmann machine, but is still useful; for example it can learn to extract interesting features from images.

⁵We thank Geoffrey Hinton for assistance in the preparation of the material on RBMs.

By alternately sampling the variables in each layer of the RBM shown in Figure 17.6, it is possible to generate samples from the joint density model. If the $\mathcal{V}_1$ part of the visible layer is clamped at a particular feature vector during the alternating sampling, it is possible to sample from the distribution over labels given $\mathcal{V}_1$. Alternatively classification of test items can also be achieved by comparing the unnormalized joint densities of each label category with the observed features. We do not need to compute the partition function as it is the same for all of these combinations.

As noted the restricted Boltzmann machine has the same generic form as a single hidden layer neural network (Section 11.3). The edges in the latter model are directed, the hidden units are usually real-valued, and the fitting criterion is different. The neural network minimizes the error (cross-entropy) between the targets and their model predictions, conditional on the input features. In contrast, the restricted Boltzmann machine maximizes the log-likelihood for the joint distribution of all visible units—that is, the features and targets. It can extract information from the input features that is useful for predicting the labels, but, unlike supervised learning methods, it may also use some of its hidden units to model structure in the feature vectors that is not immediately relevant for predicting the labels. These features may turn out to be useful, however, when combined with features derived from other hidden layers.

Unfortunately, Gibbs sampling in a restricted Boltzmann machine can be very slow, as it can take a long time to reach stationarity. As the network weights get larger, the chain mixes more slowly and we need to run more steps to get the unconditional estimates. Hinton (2002) noticed empirically that learning still works well if we estimate the second expectation in (17.37) by starting the Markov chain at the data and only running for a few steps (instead of to convergence). He calls this *contrastive divergence*: we sample $\mathcal{H}$ given $\mathcal{V}_1, \mathcal{V}_2$, then $\mathcal{V}_1, \mathcal{V}_2$ given $\mathcal{H}$ and finally $\mathcal{H}$ given $\mathcal{V}_1, \mathcal{V}_2$ again. The idea is that when the parameters are far from the solution, it may be wasteful to iterate the Gibbs sampler to stationarity, as just a single iteration will reveal a good direction for moving the estimates.

We now give an example to illustrate the use of an RBM. Using contrastive divergence, it is possible to train an RBM to recognize hand-written digits from the MNIST dataset (LeCun et al., 1998). With 2000 hidden units, 784 visible units for representing binary pixel intensities and one 10-way multinomial visible unit for representing labels, the RBM achieves an error rate of 1.9% on the test set. This is a little higher than the 1.4% achieved by a support vector machine and comparable to the error rate achieved by a neural network trained with backpropagation. The error rate of the RBM, however, can be reduced to 1.25% by replacing the 784 pixel intensities by 500 features that are produced from the images without using any label information. First, an RBM with 784 visible units and 500 hidden units is trained, using contrastive divergence, to model the set of images. Then the hidden states of the first RBM are used as data for training a

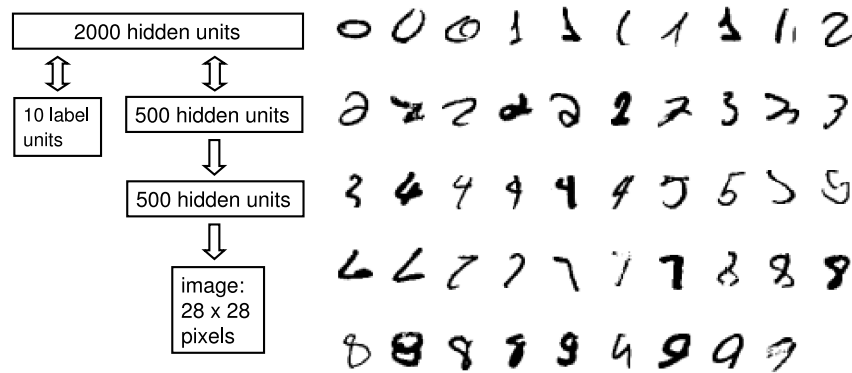


FIGURE 17.7. Example of a restricted Boltzmann machine for handwritten digit classification. The network is depicted in the schematic on the left. Displayed on the right are some difficult test images that the model classifies correctly.

second RBM that has 500 visible units and 500 hidden units. Finally, the hidden states of the second RBM are used as the features for training an RBM with 2000 hidden units as a joint density model. The details and justification for learning features in this greedy, layer-by-layer way are described in Hinton et al. (2006). Figure 17.7 gives a representation of the composite model that is learned in this way and also shows some examples of the types of distortion that it can cope with.

Bibliographic Notes

Much work has been done in defining and understanding the structure of graphical models. Comprehensive treatments of graphical models can be found in Whittaker (1990), Lauritzen (1996), Cox and Wermuth (1996), Edwards (2000), Pearl (2000), Anderson (2003), Jordan (2004), and Koller and Friedman (2007). Wasserman (2004) gives a brief introduction, and Chapter 8 of Bishop (2006) gives a more detailed overview. Boltzmann machines were proposed in Ackley et al. (1985). Ripley (1996) has a detailed chapter on topics in graphical models that relate to machine learning. We found this particularly useful for its discussion of Boltzmann machines.

Exercises

Ex. 17.1 For the Markov graph of Figure 17.8, list all of the implied conditional independence relations and find the maximal cliques.

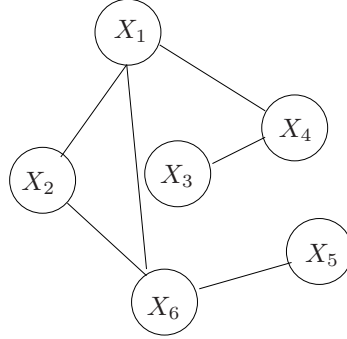


FIGURE 17.8.

Ex. 17.2 Consider random variables X_1, X_2, X_3, X_4 . In each of the following cases draw a graph that has the given independence relations:

- $X_1 \perp X_3 | X_2$ and $X_2 \perp X_4 | X_3$.
- $X_1 \perp X_4 | X_2, X_3$ and $X_2 \perp X_4 | X_1, X_3$.
- $X_1 \perp X_4 | X_2, X_3$, $X_1 \perp X_3 | X_2, X_4$ and $X_3 \perp X_4 | X_1, X_2$.

Ex. 17.3 Let Σ be the covariance matrix of a set of p variables X . Consider the partial covariance matrix $\Sigma_{a.b} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$ between the two subsets of variables $X_a = (X_1, X_2)$ consisting of the first two, and X_b the rest. This is the covariance matrix between these two variables, after linear adjustment for all the rest. In the Gaussian distribution, this is the covariance matrix of the conditional distribution of $X_a | X_b$. The partial correlation coefficient $\rho_{jk|\text{rest}}$ between the pair X_a conditional on the rest X_b , is simply computed from this partial covariance. Define $\Theta = \Sigma^{-1}$.

- Show that $\Sigma_{a.b} = \Theta_{aa}^{-1}$.
- Show that if any off-diagonal element of Θ is zero, then the partial correlation coefficient between the corresponding variables is zero.
- Show that if we treat Θ as if it were a covariance matrix, and compute the corresponding “correlation” matrix

$$\mathbf{R} = \text{diag}(\Theta)^{-1/2} \cdot \Theta \cdot \text{diag}(\Theta)^{-1/2}, \quad (17.40)$$

then $r_{jk} = -\rho_{jk|\text{rest}}$

Ex. 17.4 Denote by

$$f(X_1 | X_2, X_3, \dots, X_p)$$

the conditional density of X_1 given $X_2, \dots, X_p$. If

$$f(X_1 | X_2, X_3, \dots, X_p) = f(X_1 | X_3, \dots, X_p),$$

show that $X_1 \perp X_2 | X_3, \dots, X_p$.

Ex. 17.5 Consider the setup in Section 17.3.1 with no missing edges. Show that

$$\mathbf{S}_{11}\beta - s_{12} = 0$$

are the estimating equations for the multiple regression coefficients of the last variable on the rest.

Ex. 17.6 *Recovery of $\hat{\Theta} = \hat{\Sigma}^{-1}$ from Algorithm 17.1.* Use expression (17.16) to derive the standard partitioned inverse expressions

$$\theta_{12} = -\mathbf{W}_{11}^{-1}w_{12}\theta_{22} \quad (17.41)$$

$$\theta_{22} = 1/(w_{22} - w_{12}^T \mathbf{W}_{11}^{-1} w_{12}). \quad (17.42)$$

Since $\hat{\beta} = \mathbf{W}_{11}^{-1}w_{12}$, show that $\hat{\theta}_{22} = 1/(w_{22} - w_{12}^T \hat{\beta})$ and $\hat{\theta}_{12} = -\hat{\beta}\hat{\theta}_{22}$. Thus $\hat{\theta}_{12}$ is a simply rescaling of $\hat{\beta}$ by $-\hat{\theta}_{22}$.

Ex. 17.7 Write a program to implement the modified regression procedure in Algorithm 17.1 for fitting the Gaussian graphical model with pre-specified edges missing. Test it on the flow cytometry data from the book website, using the graph of Figure 17.1.

Ex. 17.8

- (a) Write a program to fit the lasso using the coordinate descent procedure (17.26). Compare its results to those from the `lars` program or some other convex optimizer, to check that it is working correctly.
- (b) Using the program from (a), write code to implement the graphical lasso (Algorithm 17.2). Apply it to the flow cytometry data from the book website. Vary the regularization parameter and examine the resulting networks.

Ex. 17.9 Suppose that we have a Gaussian graphical model in which some or all of the data at some vertices are missing.

- (a) Consider the EM algorithm for a dataset of N i.i.d. multivariate observations $x_i \in \mathbb{R}^p$ with mean μ and covariance matrix Σ . For each sample i , let o_i and m_i index the predictors that are observed and missing, respectively. Show that in the E step, the observations are imputed from the current estimates of μ and Σ :

$$\hat{x}_{i,m_i} = \mathbb{E}(x_{i,m_i} | x_{i,o_i}, \theta) = \hat{\mu}_{m_i} + \hat{\Sigma}_{m_i,o_i} \hat{\Sigma}_{o_i,o_i}^{-1} (x_{i,o_i} - \hat{\mu}_{o_i}) \quad (17.43)$$

while in the M step, μ and Σ are re-estimated from the empirical mean and (modified) covariance of the imputed data:

$$\hat{\mu}_j = \sum_{i=1}^N \hat{x}_{ij} / N$$

$$\hat{\Sigma}_{jj'} = \sum_{i=1}^N [(\hat{x}_{ij} - \hat{\mu}_j)(\hat{x}_{ij'} - \hat{\mu}_{j'}) + c_{i,jj'}] / N \quad (17.44)$$

where $c_{i,jj'} = \hat{\Sigma}_{jj'}$ if $j, j' \in m_i$ and zero otherwise. Explain the reason for the correction term $c_{i,jj'}$ (Little and Rubin, 2002).

- (b) Implement the EM algorithm for the Gaussian graphical model using the modified regression procedure from Exercise 17.7 for the M-step.
- (c) For the flow cytometry data on the book website, set the data for the last protein **Jnk** in the first 1000 observations to missing, fit the model of Figure 17.1, and compare the predicted values to the actual values for **Jnk**. Compare the results to those obtained from a regression of **Jnk** on the other vertices with edges to **Jnk** in Figure 17.1, using only the non-missing data.

Ex. 17.10 Using a simple binary graphical model with just two variables, show why it is essential to include a constant node $X_0 \equiv 1$ in the model.

Ex. 17.11 Show that the Ising model (17.28) for the joint probabilities in a discrete graphical model implies that the conditional distributions have the logistic form (17.30).

Ex. 17.12 Consider a Poisson regression problem with p binary variables x_{ij} , $j = 1, \dots, p$ and response variable y_i which measures the number of observations with predictor $x_i \in \{0, 1\}^p$. The design is balanced, in that all $n = 2^p$ possible combinations are measured. We assume a log-linear model for the Poisson mean in each cell

$$\log \mu(X) = \theta_{00} + \sum_{(j,k) \in E} x_{ij} x_{ik} \theta_{jk}, \quad (17.45)$$

using the same notation as in Section 17.4.1 (including the constant variable $x_{i0} = 1 \forall i$). We assume the response is distributed as

$$\Pr(Y = y | X = x) = \frac{e^{-\mu(x)} \mu(x)^y}{y!}. \quad (17.46)$$

Write down the conditional log-likelihood for the observed responses y_i , and compute the gradient.

- (a) Show that the gradient equation for θ_{00} computes the partition function (17.29).
- (b) Show that the gradient equations for the remainder of the parameters are equivalent to the gradient (17.34).

18

High-Dimensional Problems: $p \gg N$

18.1 When p is Much Bigger than N

In this chapter we discuss prediction problems in which the number of features p is much larger than the number of observations N , often written $p \gg N$. Such problems have become of increasing importance, especially in genomics and other areas of computational biology. We will see that high variance and overfitting are a major concern in this setting. As a result, simple, highly regularized approaches often become the methods of choice. The first part of the chapter focuses on prediction in both the classification and regression settings, while the second part discusses the more basic problem of feature selection and assessment.

To get us started, Figure 18.1 summarizes a small simulation study that demonstrates the “less fitting is better” principle that applies when $p \gg N$. For each of $N = 100$ samples, we generated p standard Gaussian features X with pairwise correlation 0.2. The outcome Y was generated according to a linear model

$$Y = \sum_{j=1}^p X_j \beta_j + \sigma \varepsilon \quad (18.1)$$

where ε was generated from a standard Gaussian distribution. For each dataset, the set of coefficients β_j were also generated from a standard Gaussian distribution. We investigated three cases: $p = 20, 100$, and 1000 . The standard deviation σ was chosen in each case so that the signal-to-noise ratio $\text{Var}[E(Y|X)]/\sigma^2$ equaled 2. As a result, the number of significant uni-

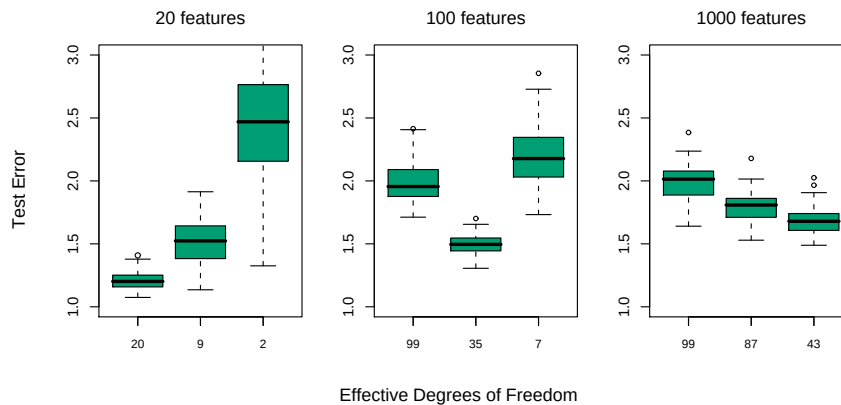


FIGURE 18.1. Test-error results for simulation experiments. Shown are boxplots of the relative test errors over 100 simulations, for three different values of p , the number of features. The relative error is the test error divided by the Bayes error, σ^2 . From left to right, results are shown for ridge regression with three different values of the regularization parameter λ : 0.001, 100 and 1000. The (average) effective degrees of freedom in the fit is indicated below each plot.

variate regression coefficients¹ was 9, 33 and 331, respectively, averaged over the 100 simulation runs. The $p = 1000$ case is designed to mimic the kind of data that we might see in a high-dimensional genomic or proteomic dataset, for example.

We fit a ridge regression to the data, with three different values for the regularization parameter λ : 0.001, 100, and 1000. When $\lambda = 0.001$, this is nearly the same as least squares regression, with a little regularization just to ensure that the problem is non-singular when $p > N$. Figure 18.1 shows boxplots of the relative test error achieved by the different estimators in each scenario. The corresponding average degrees of freedom used in each ridge-regression fit is indicated (computed using formula (3.50) on page 68²). The degrees of freedom is a more interpretable parameter than λ . We see that ridge regression with $\lambda = 0.001$ (20 df) wins when $p = 20$; $\lambda = 100$ (35 df) wins when $p = 100$, and $\lambda = 1000$ (43 df) wins when $p = 1000$.

Here is an explanation for these results. When $p = 20$, we fit all the way and we can identify as many of the significant coefficients as possible with

¹We call a regression coefficient significant if $|\hat{\beta}_j/\widehat{\text{se}}_j| \geq 2$, where $\hat{\beta}_j$ is the estimated (univariate) coefficient and $\widehat{\text{se}}_j$ is its estimated standard error.

²For a fixed value of the regularization parameter λ , the degrees of freedom depends on the observed predictor values in each simulation. Hence we compute the average degrees of freedom over simulations.

low bias. When $p = 100$, we can identify some non-zero coefficients using moderate shrinkage. Finally, when $p = 1000$, even though there are many nonzero coefficients, we don't have a hope for finding them and we need to shrink all the way down. As evidence of this, let $t_j = \hat{\beta}_j / \hat{se}_j$, where $\hat{\beta}_j$ is the ridge regression estimate and $\hat{se}_j$ its estimated standard error. Then using the optimal ridge parameter in each of the three cases, the median value of $|t_j|$ was 2.0, 0.6 and 0.2, and the average number of $|t_j|$ values exceeding 2 was equal to 9.8, 1.2 and 0.0.

Ridge regression with $\lambda = 0.001$ successfully exploits the correlation in the features when $p < N$, but cannot do so when $p \gg N$. In the latter case there is not enough information in the relatively small number of samples to efficiently estimate the high-dimensional covariance matrix. In that case, more regularization leads to superior prediction performance.

Thus it is not surprising that the analysis of high-dimensional data requires either modification of procedures designed for the $N > p$ scenario, or entirely new procedures. In this chapter we discuss examples of both kinds of approaches for high dimensional classification and regression; these methods tend to regularize quite heavily, using scientific contextual knowledge to suggest the appropriate form for this regularization. The chapter ends with a discussion of feature selection and multiple testing.

18.2 Diagonal Linear Discriminant Analysis and Nearest Shrunk Centroids

Gene expression arrays are an important new technology in biology, and are discussed in Chapters 1 and 14. The data in our next example form a matrix of 2308 genes (columns) and 63 samples (rows), from a set of microarray experiments. Each expression value is a log-ratio $\log(R/G)$. R is the amount of gene-specific RNA in the target sample that hybridizes to a particular (gene-specific) spot on the microarray, and G is the corresponding amount of RNA from a reference sample. The samples arose from small, round blue-cell tumors (SRBCT) found in children, and are classified into four major types: BL (Burkitt lymphoma), EWS (Ewing's sarcoma), NB (neuroblastoma), and RMS (rhabdomyosarcoma). There is an additional test data set of 20 observations. We will not go into the scientific background here.

Since $p \gg N$, we cannot fit a full linear discriminant analysis (LDA) to the data; some sort of regularization is needed. The method we describe here is similar to the methods of Section 4.3.1, but with important modifications that achieve feature selection. The simplest form of regularization assumes that the features are independent within each class, that is, the within-class covariance matrix is diagonal. Despite the fact that features will rarely be independent within a class, when $p \gg N$ we don't have

enough data to estimate their dependencies. The assumption of independence greatly reduces the number of parameters in the model and often results in an effective and interpretable classifier.

Thus we consider the *diagonal-covariance* LDA rule for classifying the classes. The *discriminant score* [see (4.12) on page 110] for class k is

$$\delta_k(x^*) = - \sum_{j=1}^p \frac{(x_j^* - \bar{x}_{kj})^2}{s_j^2} + 2 \log \pi_k. \quad (18.2)$$

Here $x^* = (x_1^*, x_2^*, \dots, x_p^*)^T$ is a vector of expression values for a test observation, s_j is the pooled within-class standard deviation of the j th gene, and $\bar{x}_{kj} = \sum_{i \in C_k} x_{ij} / N_k$ is the mean of the N_k values for gene j in class k , with C_k being the index set for class k . We call $\tilde{x}_k = (\bar{x}_{k1}, \bar{x}_{k2}, \dots, \bar{x}_{kp})^T$ the *centroid* of class k . The first part of (18.2) is simply the (negative) standardized squared distance of x^* to the k th centroid. The second part is a correction based on the class *prior probability* π_k , where $\sum_{k=1}^K \pi_k = 1$. The classification rule is then

$$C(x^*) = \ell \text{ if } \delta_\ell(x^*) = \max_k \delta_k(x^*). \quad (18.3)$$

We see that the diagonal LDA classifier is equivalent to a nearest centroid classifier after appropriate standardization. It is also a special case of the naive-Bayes classifier, as described in Section 6.6.3. It assumes that the features in each class have independent Gaussian distributions with the same variance.

The diagonal LDA classifier is often effective in high dimensional settings. It is also called the “independence rule” in Bickel and Levina (2004), who demonstrate theoretically that it will often outperform standard linear discriminant analysis in high-dimensional problems. Here the diagonal LDA classifier yielded five misclassification errors for the 20 test samples. One drawback of the diagonal LDA classifier is that it uses all of the features (genes), and hence is not convenient for interpretation. With further regularization we can do better—both in terms of test error and interpretability.

We would like to regularize in a way that automatically drops out features that are not contributing to the class predictions. We can do this by shrinking the classwise mean toward the overall mean, for each feature separately. The result is a regularized version of the nearest centroid classifier, or equivalently a regularized version of the diagonal-covariance form of LDA. We call the procedure *nearest shrunken centroids* (NSC).

The shrinkage procedure is defined as follows. Let

$$d_{kj} = \frac{\bar{x}_{kj} - \bar{x}_j}{m_k(s_j + s_0)}, \quad (18.4)$$

where $\bar{x}_j$ is the overall mean for gene j , $m_k^2 = 1/N_k - 1/N$ and s_0 is a small positive constant, typically chosen to be the median of the s_j values.

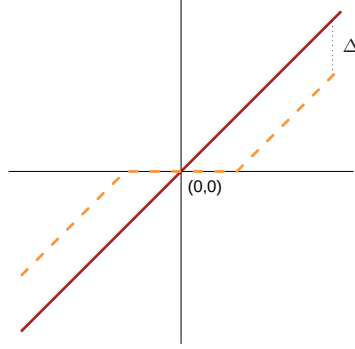


FIGURE 18.2. Soft thresholding function $\text{sign}(x)(|x| - \Delta)_+$ is shown in orange, along with the 45° line in red.

This constant guards against large d_{kj} values that arise from expression values near zero. With constant within-class variance σ^2 , the variance of the contrast $\bar{x}_{kj} - \bar{x}_j$ in the numerator is $m_k^2 \sigma^2$, and hence the form of the standardization in the denominator. We shrink the d_{kj} toward zero using soft thresholding

$$d'_{kj} = \text{sign}(d_{kj})(|d_{kj}| - \Delta)_+; \quad (18.5)$$

see Figure 18.2. Here Δ is a parameter to be determined; we used 10-fold cross-validation in the example (see the top panel of Figure 18.4). Each d_{kj} is reduced by an amount Δ in absolute value, and is set to zero if its value is less than zero. The soft-thresholding function is shown in Figure 18.2; the same thresholding is applied to wavelet coefficients in Section 5.9. An alternative is to use hard thresholding

$$d'_{kj} = d_{kj} \cdot I(|d_{kj}| \geq \Delta); \quad (18.6)$$

we prefer soft-thresholding, as it is a smoother operation and typically works better. The shrunken versions of $\bar{x}_{kj}$ are then obtained by reversing the transformation in (18.4):

$$\bar{x}'_{kj} = \bar{x}_j + m_k(s_j + s_0)d'_{kj}. \quad (18.7)$$

We then use the shrunken centroids $\bar{x}'_{kj}$ in place of the original $\bar{x}_{kj}$ in the discriminant score (18.2). The estimator (18.7) can also be viewed as a lasso-style estimator for the class means (Exercise 18.2).

Notice that only the genes that have a nonzero d'_{kj} for at least one of the classes play a role in the classification rule, and hence the vast majority of genes can often be discarded. In this example, all but 43 genes were discarded, leaving a small interpretable set of genes that characterize each class. Figure 18.3 represents the genes in a heatmap.

Figure 18.4 (top panel) demonstrates the effectiveness of the shrinkage. With no shrinkage we make 5/20 errors on the test data, and several errors

on the training and CV data. The shrunk centroids achieve zero test errors for a fairly broad band of values for Δ . The bottom panel of Figure 18.4 shows the four centroids for the SRBCT data (gray), relative to the overall centroid. The blue bars are shrunk versions of these centroids, obtained by soft-thresholding the gray bars, using $\Delta = 4.3$. The discriminant scores (18.2) can be used to construct class probability estimates:

$$\hat{p}_k(x^*) = \frac{e^{\frac{1}{2}\delta_k(x^*)}}{\sum_{\ell=1}^K e^{\frac{1}{2}\delta_\ell(x^*)}}. \quad (18.8)$$

These can be used to rate the classifications, or to decide not to classify a particular sample at all.

Note that other forms of feature selection can be used in this setting, including hard thresholding. Fan and Fan (2008) show theoretically the importance of carrying out some kind of feature selection with diagonal linear discriminant analysis in high-dimensional problems.

18.3 Linear Classifiers with Quadratic Regularization

Ramaswamy et al. (2001) present a more difficult microarray classification problem, involving a training set of 144 patients with 14 different types of cancer, and a test set of 54 patients. Gene expression measurements were available for 16,063 genes.

Table 18.1 shows the prediction results from eight different classification methods. The data from each patient was first standardized to have mean 0 and variance 1; this seems to improve prediction accuracy overall this example, suggesting that the “shape” of each gene-expression profile is important, rather than the absolute expression levels. In each case, the

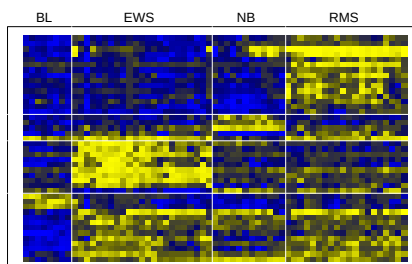


FIGURE 18.3. Heat-map of the chosen 43 genes. Within each of the horizontal partitions, we have ordered the genes by hierarchical clustering, and similarly for the samples within each vertical partition. Yellow represents over- and blue under-expression.

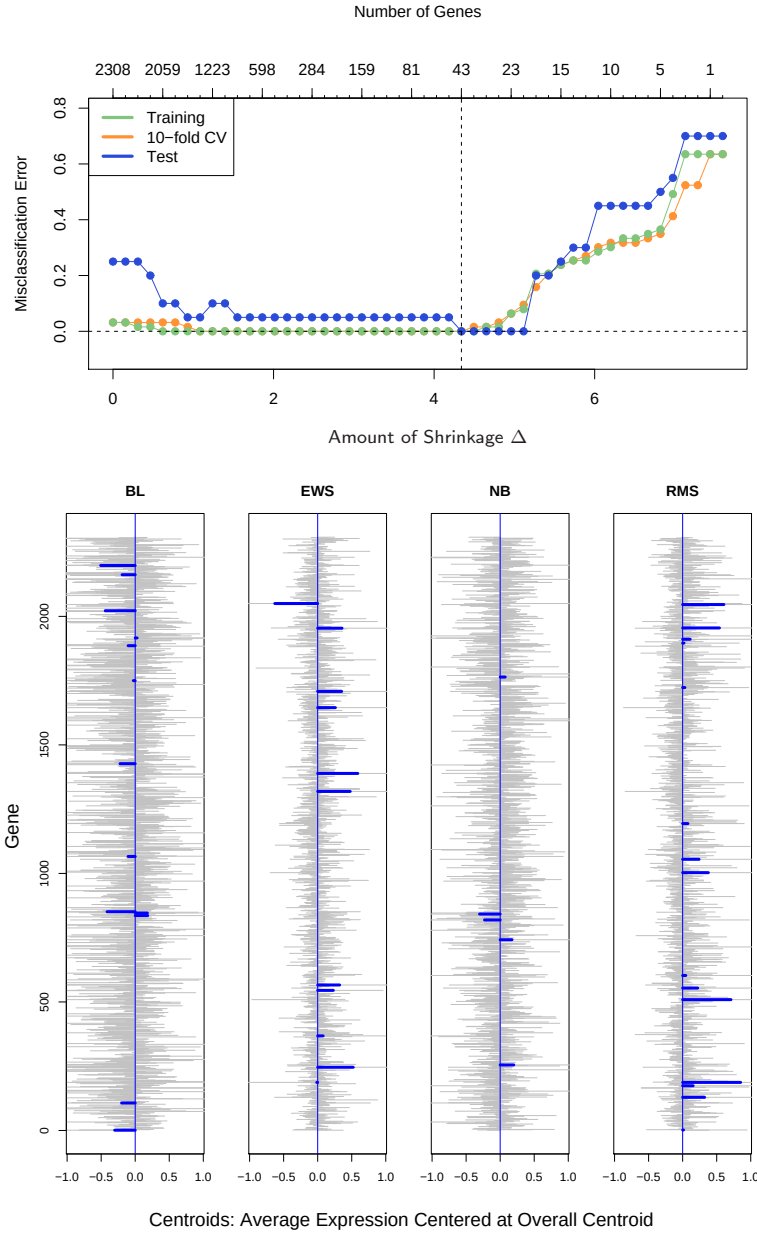


FIGURE 18.4. (Top): Error curves for the SRBCT data. Shown are the training, 10-fold cross-validation, and test misclassification errors as the threshold parameter Δ is varied. The value $\Delta = 4.34$ is chosen by CV, resulting in a subset of 43 selected genes. (Bottom): Four centroids profiles d_{kj} for the SRBCT data (gray), relative to the overall centroid. Each centroid has 2308 components, and we see considerable noise. The blue bars are shrunk versions d'_{kj} of these centroids, obtained by soft-thresholding the gray bars, using $\Delta = 4.3$.

TABLE 18.1. *Prediction results for microarray data with 14 cancer classes. Method 1 is described in Section 18.2. Methods 2, 3 and 6 are discussed in Section 18.3, while 4, 7 and 8 are discussed in Section 18.4. Method 5 is described in Section 13.3. The elastic-net penalized multinomial does the best on the test data, but the standard error of each test-error estimate is about 3, so such comparisons are inconclusive.*

Methods	CV errors (SE) Out of 144	Test errors Out of 54	Number of Genes Used
1. Nearest shrunken centroids	35 (5.0)	17	6,520
2. L_2 -penalized discriminant analysis	25 (4.1)	12	16,063
3. Support vector classifier	26 (4.2)	14	16,063
4. Lasso regression (one vs all)	30.7 (1.8)	12.5	1,429
5. k -nearest neighbors	41 (4.6)	26	16,063
6. L_2 -penalized multinomial	26 (4.2)	15	16,063
7. L_1 -penalized multinomial	17 (2.8)	13	269
8. Elastic-net penalized multinomial	22 (3.7)	11.8	384

regularization parameter has been chosen to minimize the cross-validation error, and the test error at that value of the parameter is shown. When more than one value of the regularization parameter yields the minimal cross-validation error, the average test error at these values is reported.

RDA (regularized discriminant analysis), regularized multinomial logistic regression, and the support vector machine are more complex methods that try to exploit multivariate information in the data. We describe each in turn, as well as a variety of regularization methods, including both L_1 and L_2 and some in between.

18.3.1 Regularized Discriminant Analysis

Regularized discriminant analysis (RDA) is described in Section 4.3.1. Linear discriminant analysis involves the inversion of a $p \times p$ within-covariance matrix. When $p \gg N$, this matrix can be huge, has rank at most $N < p$, and hence is singular. RDA overcomes the singularity issues by regularizing the within-covariance estimate $\hat{\Sigma}$. Here we use a version of RDA that shrinks $\hat{\Sigma}$ towards its diagonal:

$$\hat{\Sigma}(\gamma) = \gamma \hat{\Sigma} + (1 - \gamma) \text{diag}(\hat{\Sigma}), \text{ with } \gamma \in [0, 1]. \quad (18.9)$$

Note that $\gamma = 0$ corresponds to diagonal LDA, which is the “no shrinkage” version of nearest shrunken centroids. The form of shrinkage in (18.9) is

much like ridge regression (Section 3.4.1), which shrinks the total covariance matrix of the features towards a diagonal (scalar) matrix. In fact, viewing linear discriminant analysis as linear regression with optimal scoring of the categorical response (see (12.57) in Section 12.6), the equivalence becomes more precise.

The computational burden of inverting this large $p \times p$ matrix is overcome using the methods discussed in Section 18.3.5. The value of γ was chosen by cross-validation in line 2 of Table 18.1; all values of $\gamma \in (0.002, 0.550)$ gave the same CV and test error. Further development of RDA, including shrinkage of the centroids in addition to the covariance matrix, can be found in Guo et al. (2006).

18.3.2 Logistic Regression with Quadratic Regularization

Logistic regression (Section 4.4) can be modified in a similar way, to deal with the $p \gg N$ case. With K classes, we use a symmetric version of the multiclass logistic model (4.17) on page 119:

$$\Pr(G = k | X = x) = \frac{\exp(\beta_{k0} + x^T \beta_k)}{\sum_{\ell=1}^K \exp(\beta_{\ell 0} + x^T \beta_{\ell})}. \quad (18.10)$$

This has K coefficient vectors of log-odds parameters $\beta_1, \beta_2, \dots, \beta_K$. We regularize the fitting by maximizing the penalized log-likelihood

$$\max_{\{\beta_{0k}, \beta_k\}_1^K} \left[\sum_{i=1}^N \log \Pr(g_i | x_i) - \frac{\lambda}{2} \sum_{k=1}^K \|\beta_k\|_2^2 \right]. \quad (18.11)$$

This regularization automatically resolves the redundancy in the parameterization, and forces $\sum_{k=1}^K \hat{\beta}_{kj} = 0$, $j = 1, \dots, p$ (Exercise 18.3). Note that the constant terms β_{k0} are not regularized (and so one should be set to zero). The resulting optimization problem is convex, and can be solved by a Newton algorithm or other numerical techniques. Details are given in Zhu and Hastie (2004). Friedman et al. (2010) provide software for computing the regularization path for the two- and multiclass logistic regression models. Table 18.1, line 6 reports the results for the multiclass logistic regression model, referred to there as “multinomial”. It can be shown (Rosset et al., 2004a) that for separable data, as $\lambda \rightarrow 0$, the regularized (two-class) logistic regression estimate (renormalized) converges to the maximal margin classifier (Section 12.2). This gives an attractive alternative to the support-vector machine, discussed next, especially in the multiclass case.

18.3.3 The Support Vector Classifier

The support vector classifier is described for the two-class case in Section 12.2. When $p > N$, it is especially attractive because in general the

classes are perfectly separable by a hyperplane unless there are identical feature vectors in different classes. Without any regularization the support vector classifier finds the separating hyperplane with the largest margin; that is, the hyperplane yielding the biggest gap between the classes in the training data. Somewhat surprisingly, when $p \gg N$ the unregularized support vector classifier often works about as well as the best regularized version. Overfitting often does not seem to be a problem, partly because of the insensitivity of misclassification loss.

There are many different methods for generalizing the two-class support-vector classifier to $K > 2$ classes. In the “one versus one” (OVO) approach, we compute all $\binom{K}{2}$ pairwise classifiers. For each test point, the predicted class is the one that wins the most pairwise contests. In the “one versus all” (OVA) approach, each class is compared to all of the others in K two-class comparisons. To classify a test point, we compute the confidences (signed distance from the hyperplane) for each of the K classifiers. The winner is the class with the highest confidence. Finally, Vapnik (1998) and Weston and Watkins (1999) suggested (somewhat complex) multiclass criteria which generalize the two-class criterion (12.7).

Tibshirani and Hastie (2007) propose the *margin tree* classifier, in which support-vector classifiers are used in a binary tree, much as in CART (Chapter 9). The classes are organized in a hierarchical manner, which can be useful for classifying patients into different cancer types, for example.

Line 3 of Table 18.1 shows the results for the support vector classifier using the OVA method; Ramaswamy et al. (2001) reported (and we confirmed) that this approach worked best for this problem. The errors are very similar to those in line 6, as we might expect from the comments at the end of the previous section. The error rates are insensitive to the choice of C [the regularization parameter in (12.8) on page 420], for values of $C > 0.001$. Since $p > N$, the support vector hyperplane can perfectly separate the training data by setting $C = \infty$.

18.3.4 Feature Selection

Feature selection is an important scientific requirement for a classifier when p is large. Neither discriminant analysis, logistic regression, nor the support-vector classifier perform feature selection automatically, because all use quadratic regularization. All features have nonzero weights in both models. Ad-hoc methods for feature selection have been proposed, for example, removing genes with small coefficients, and refitting the classifier. This is done in a backward stepwise manner, starting with the smallest weights and moving on to larger weights. This is known as *recursive feature elimination* (Guyon et al., 2002). It was not successful in this example; Ramaswamy et al. (2001) report, for example, that the accuracy of the support-vector classifier starts to degrade as the number of genes is reduced from the full

set of 16,063. This is rather remarkable, as the number of training samples is only 144. We do not have an explanation for this behavior.

All three methods discussed in this section (RDA, LR and SVM) can be modified to fit nonlinear decision boundaries using kernels. Usually the motivation for such an approach is to increase the model complexity. With $p \gg N$ the models are already sufficiently complex and overfitting is always a danger. Yet despite the high dimensionality, radial kernels (Section 12.3.3) sometimes deliver superior results in these high dimensional problems. The radial kernel tends to dampen inner products between points far away from each other, which in turn leads to robustness to outliers. This occurs often in high dimensions, and may explain the positive results. We tried a radial kernel with the SVM in Table 18.1, but in this case the performance was inferior.

18.3.5 Computational Shortcuts When $p \gg N$

The computational techniques discussed in this section apply to any method that fits a linear model with quadratic regularization on the coefficients. That includes all the methods discussed in this section, and many more. When $p > N$, the computations can be carried out in an N -dimensional space, rather than p , via the singular value decomposition introduced in Section 14.5. Here is the geometric intuition: just like two points in three-dimensional space always lie on a line, N points in p -dimensional space lie in an $(N - 1)$ -dimensional affine subspace.

Given the $N \times p$ data matrix $\mathbf{X}$, let

$$\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T \quad (18.12)$$

$$= \mathbf{R}\mathbf{V}^T \quad (18.13)$$

be the singular-value decomposition (SVD) of $\mathbf{X}$; that is, $\mathbf{V}$ is $p \times N$ with orthonormal columns, $\mathbf{U}$ is $N \times N$ orthogonal, and $\mathbf{D}$ a diagonal matrix with elements $d_1 \geq d_2 \geq \dots \geq d_N \geq 0$. The matrix $\mathbf{R}$ is $N \times N$, with rows r_i^T .

As a simple example, let's first consider the estimates from a ridge regression:

$$\hat{\beta} = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}. \quad (18.14)$$

Replacing $\mathbf{X}$ by $\mathbf{R}\mathbf{V}^T$ and after some further manipulations, this can be shown to equal

$$\hat{\beta} = \mathbf{V}(\mathbf{R}^T\mathbf{R} + \lambda\mathbf{I})^{-1}\mathbf{R}^T\mathbf{y} \quad (18.15)$$

(Exercise 18.4). Thus $\hat{\beta} = \mathbf{V}\hat{\theta}$, where $\hat{\theta}$ is the ridge-regression estimate using the N observations (r_i, y_i) , $i = 1, 2, \dots, N$. In other words, we can simply reduce the data matrix from $\mathbf{X}$ to $\mathbf{R}$, and work with the rows of $\mathbf{R}$. This trick reduces the computational cost from $O(p^3)$ to $O(pN^2)$ when $p > N$.

These results can be generalized to *all* models that are linear in the parameters and have quadratic penalties. Consider any supervised learning problem where we use a linear function $f(X) = \beta_0 + X^T \beta$ to model a parameter in the conditional distribution of $Y|X$. We fit the parameters β by minimizing some loss function $\sum_{i=1}^N L(y_i, f(x_i))$ over the data with a quadratic penalty on β . Logistic regression is a useful example to have in mind. Then we have the following simple theorem:

Let $f^*(r_i) = \theta_0 + r_i^T \theta$ with r_i defined in (18.13), and consider the pair of optimization problems:

$$(\hat{\beta}_0, \hat{\beta}) = \arg \min_{\beta_0, \beta \in \mathbb{R}^p} \sum_{i=1}^N L(y_i, \beta_0 + x_i^T \beta) + \lambda \beta^T \beta; \quad (18.16)$$

$$(\hat{\theta}_0, \hat{\theta}) = \arg \min_{\theta_0, \theta \in \mathbb{R}^N} \sum_{i=1}^N L(y_i, \theta_0 + r_i^T \theta) + \lambda \theta^T \theta. \quad (18.17)$$

Then the $\hat{\beta}_0 = \hat{\theta}_0$, and $\hat{\beta} = \mathbf{V} \hat{\theta}$.

The theorem says that we can simply replace the p vectors x_i by the N -vectors r_i , and perform our penalized fit as before, but with far fewer predictors. The N -vector solution $\hat{\theta}$ is then transformed back to the p -vector solution via a simple matrix multiplication. This result is part of the statistics folklore, and deserves to be known more widely—see Hastie and Tibshirani (2004) for further details.

Geometrically, we are rotating the features to a coordinate system in which all but the first N coordinates are zero. Such rotations are allowed since the quadratic penalty is invariant under rotations, and linear models are equivariant.

This result can be applied to many of the learning methods discussed in this chapter, such as regularized (multiclass) logistic regression, linear discriminant analysis (Exercise 18.6), and support vector machines. It also applies to neural networks with quadratic regularization (Section 11.5.2). Note, however, that it does not apply to methods such as the lasso, which uses nonquadratic (L_1) penalties on the coefficients.

Typically we use cross-validation to select the parameter λ . It can be seen (Exercise 18.12) that we only need to construct $\mathbf{R}$ once, on the original data, and use it as the data for each of the CV folds.

The support vector “kernel trick” of Section 12.3.7 exploits the same reduction used in this section, in a slightly different context. Suppose we have at our disposal the $N \times N$ gram (inner-product) matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. From (18.12) we have $\mathbf{K} = \mathbf{U}\mathbf{D}^2\mathbf{U}^T$, and so $\mathbf{K}$ captures the same information as $\mathbf{R}$. Exercise 18.13 shows how we can exploit the ideas in this section to fit a ridged logistic regression with $\mathbf{K}$ using its SVD.

18.4 Linear Classifiers with L_1 Regularization

The methods of Section 18.3 use an L_2 penalty to regularize their parameters, just as in ridge regression. All of the estimated coefficients are nonzero, and hence no feature selection is performed. In this section we discuss methods that use L_1 penalties instead, and hence provide automatic feature selection.

Recall the lasso of Section 3.4.2,

$$\min_{\beta} \frac{1}{2} \sum_{i=1}^N \left(y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j \right)^2 + \lambda \sum_{j=1}^p |\beta_j|, \quad (18.18)$$

which we have written in the Lagrange form (3.52). As discussed there, the use of the L_1 penalty causes a subset of the solution coefficients $\hat{\beta}_j$ to be exactly zero, for a sufficiently large value of the tuning parameter λ .

In Section 3.8.1 we discussed the LARS algorithm, an efficient procedure for computing the lasso solution for all λ . When $p > N$ (as in this chapter), as λ approaches zero, the lasso fits the training data exactly. In fact, by convex duality one can show that when $p > N$ the number of non-zero coefficients is at most N for all values of λ (Rosset and Zhu, 2007, for example). Thus the lasso provides a (severe) form of feature selection.

Lasso regression can be applied to a two-class classification problem by coding the outcome ± 1 , and applying a cutoff (usually 0) to the predictions. For more than two classes, there are many possible approaches, including the OVA and OVO methods discussed in Section 18.3.3. We tried the OVA-approach on the cancer data in Section 18.3. The results are shown in line (4) of Table 18.1. Its performance is among the best.

A more natural approach for classification problems is to use the lasso penalty to regularize logistic regression. Several implementations have been proposed in the literature, including path algorithms similar to LARS (Park and Hastie, 2007). Because the paths are piecewise smooth but nonlinear, exact methods are slower than the LARS algorithm, and are less feasible when p is large.

Friedman et al. (2010) provide very fast algorithms for fitting L_1 -penalized logistic and multinomial regression models. They use the symmetric multinomial logistic regression model as in (18.10) in Section 18.3.2, and maximize the penalized log-likelihood

$$\max_{\{\beta_{0k}, \beta_k \in \mathbb{R}^p\}_1^K} \left[\sum_{i=1}^N \log \Pr(g_i | x_i) - \lambda \sum_{k=1}^K \sum_{j=1}^p |\beta_{kj}| \right]; \quad (18.19)$$

compare with (18.11). Their algorithm computes the exact solution at a pre-chosen sequence of values for λ by cyclical coordinate descent (Section 3.8.6), and exploits the fact that solutions are sparse when $p \gg N$,

as well as the fact that solutions for neighboring values of λ tend to be very similar. This method was used in line (7) of Table 18.1, with the overall tuning parameter λ chosen by cross-validation. The performance was similar to that of the best methods, except here the automatic feature selection chose 269 genes altogether. A similar approach is used in Genkin et al. (2007); although they present their model from a Bayesian point of view, they in fact compute the posterior mode, which solves the penalized maximum-likelihood problem.

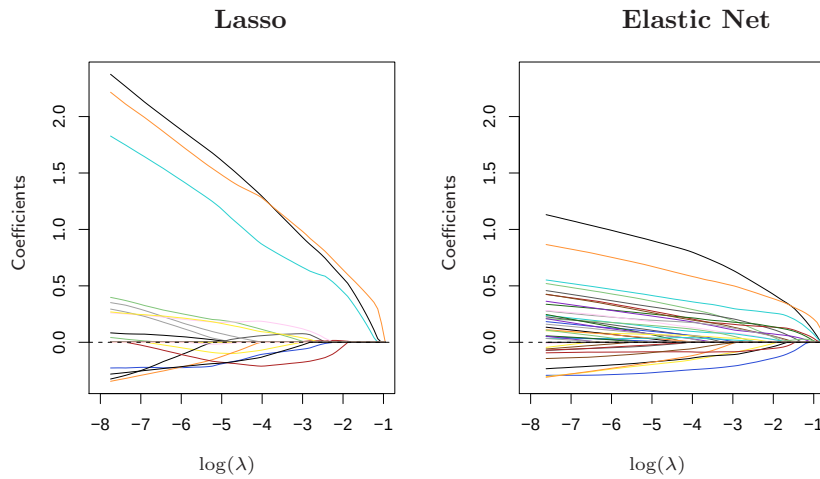


FIGURE 18.5. Regularized logistic regression paths for the leukemia data. The left panel is the lasso path, the right panel the elastic-net path with $\alpha = 0.8$. At the ends of the path (extreme left), there are 19 nonzero coefficients for the lasso, and 39 for the elastic net. The averaging effect of the elastic net results in more non-zero coefficients than the lasso, but with smaller magnitudes.

In genomic applications, there are often strong correlations among the variables; genes tend to operate in molecular pathways. The lasso penalty is somewhat indifferent to the choice among a set of strong but correlated variables (Exercise 3.28). The ridge penalty, on the other hand, tends to shrink the coefficients of correlated variables toward each other (Exercise 3.29 on page 99). The *elastic net* penalty (Zou and Hastie, 2005) is a compromise, and has the form

$$\sum_{j=1}^p (\alpha |\beta_j| + (1 - \alpha) \beta_j^2). \quad (18.20)$$

The second term encourages highly correlated features to be averaged, while the first term encourages a sparse solution in the coefficients of these aver-

aged features. The elastic net penalty can be used with any linear model, in particular for regression or classification.

Hence the multinomial problem above with elastic-net penalty becomes

$$\max_{\{\beta_{0k}, \beta_k \in \mathbb{R}^p\}_1^K} \left[\sum_{i=1}^N \log \Pr(g_i | x_i) - \lambda \sum_{k=1}^K \sum_{j=1}^p (\alpha |\beta_{kj}| + (1 - \alpha) \beta_{kj}^2) \right]. \quad (18.21)$$

The parameter α determines the mix of the penalties, and is often pre-chosen on qualitative grounds. The elastic net can yield more than N non-zero coefficients when $p > N$, a potential advantage over the lasso. Line (8) in Table 18.1 uses this model, with α and λ chosen by cross-validation. We used a sequence of 20 values of α between 0.05 and 1.0, and a 100 values of λ uniform on the log scale covering the entire range. Values of $\alpha \in [0.75, 0.80]$ gave the minimum CV error, with values of $\lambda < 0.001$ for all tied solutions. Although it has the lowest test error among all methods, the margin is small and not significant. Interestingly, when CV is performed separately for each value of α , a minimum test error of 8.8 is achieved at $\alpha = 0.10$, but this is not the value chosen in the two-dimensional CV.

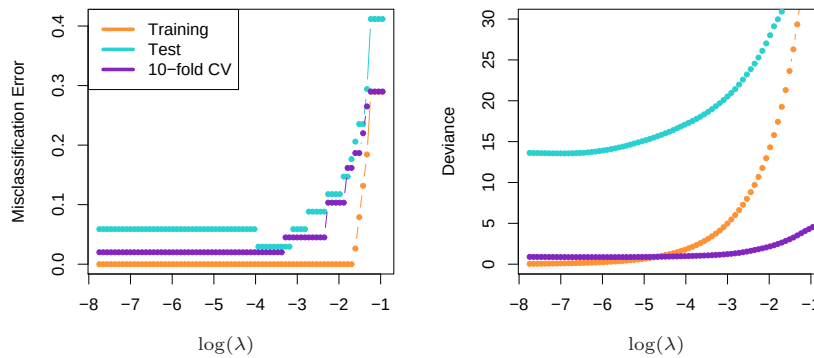


FIGURE 18.6. Training, test, and 10-fold cross validation curves for lasso logistic regression on the leukemia data. The left panel shows misclassification errors, the right panel shows deviance.

Figure 18.5 shows the lasso and elastic-net coefficient paths on the two-class leukemia data (Golub et al., 1999). There are 7129 gene-expression measurements on 38 samples, 27 of them in class ALL (acute lymphocytic leukemia), and 11 in class AML (acute myelogenous leukemia). There is also a test set with 34 samples (20, 14). Since the data are linearly separable, the solution is undefined at $\lambda = 0$ (Exercise 18.11), and degrades for very small values of λ . Hence the paths have been truncated as the fitted probabilities approach 0 and 1. There are 19 non-zero coefficients in the left plot, and 39 in the right. Figure 18.6 (left panel) shows the misclas-

sification errors for the lasso logistic regression on the training and test data, as well as for 10-fold cross-validation on the training data. The right panel uses binomial deviance to measure errors, and is much smoother. The small sample sizes lead to considerable sampling variance in these curves, even though individual curves are relatively smooth (see, for example, Figure 7.1 on page 220). Both of these plots suggest that the limiting solution $\lambda \downarrow 0$ is adequate, leading to 3/34 misclassifications in the test set. The corresponding figures for the elastic net are qualitatively similar and are not shown.

For $p \gg N$, the limiting coefficients diverge for all regularized logistic regression models, so in practical software implementations a minimum value for $\lambda > 0$ is either explicitly or implicitly set. However, renormalized versions of the coefficients converge, and these limiting solutions can be thought of as interesting alternatives to the linear optimal separating hyperplane (SVM). With $\alpha = 0$ the limiting solution coincides with the SVM (see end of Section 18.3.2), but all the 7129 genes are selected. With $\alpha = 1$, the limiting solution coincides with an L_1 separating hyperplane (Rosset et al., 2004a), and includes at most 38 genes. As α decreases from 1, the elastic-net solutions include more genes in the separating hyperplane.

18.4.1 Application of Lasso to Protein Mass Spectroscopy

Protein mass spectrometry has become a popular technology for analyzing the proteins in blood, and can be used to diagnose a disease or understand the processes underlying it.

For each blood serum sample i , we observe the intensity x_{ij} for many *time of flight* values t_j . This intensity is related to the number of particles observed to take approximately t_j time to pass from the emitter to the detector during a cycle of operation of the machine. The time of flight has a known relationship to the mass over charge ratio (m/z) of the constituent proteins in the blood. Hence the identification of a peak in the spectrum at a certain t_j tells us that there is a protein with a corresponding mass and charge. The identity of this protein can then be determined by other means.

Figure 18.7 shows an example taken from Adam et al. (2003). It shows the average spectra for healthy patients and those with prostate cancer. There are 16,898 m/z sites in total, ranging in value from 2000 to 40,000. The full dataset consists of 157 healthy patients and 167 with cancer, and the goal is to find m/z sites that discriminate between the two groups. This is an example of *functional* data; the predictors can be viewed as a function of m/z . There has been much interest in this problem in the past few years; see e.g. Petricoin et al. (2002).

The data were first standardized (baseline subtraction and normalization), and we restricted attention to m/z values between 2000 and 40,000 (spectra outside of this range were not of interest). We then applied near-

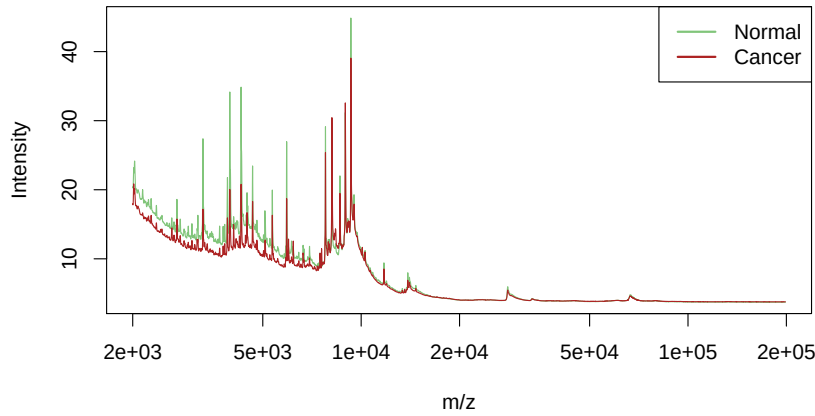


FIGURE 18.7. Protein mass spectrometry data: average profiles from normal and prostate cancer patients.

est shrunk centroids and lasso regression to the data, with the results for both methods shown in Table 18.2.

By fitting harder to the data, the lasso achieves a considerably lower test error rate. However, it may not provide a scientifically useful solution. Ideally, protein mass spectrometry resolves a biological sample into its constituent proteins, and these should appear as peaks in the spectra. The lasso doesn't treat peaks in any special way, so not surprisingly only some of the non-zero lasso weights were situated near peaks in the spectra. Furthermore, the same protein may yield a peak at slightly different m/z values in different spectra. In order to identify common peaks, some kind of m/z *warping* is needed from sample to sample.

To address this, we applied a standard peak-extraction algorithm to each spectrum, yielding a total of 5178 peaks in the 217 training spectra. Our idea was to pool the collection of peaks from all patients, and hence construct a set of common peaks. For this purpose, we applied hierarchical clustering to the positions of these peaks along the log m/z axis. We cut the resulting dendrogram horizontally at height $\log(0.005)^3$, and computed averages of the peak positions in each resulting cluster. This process yielded 728 common clusters and their corresponding peak centers.

Given these 728 common peaks, we determined which of these were present in each individual spectrum, and if present, the height of the peak. A peak height of zero was assigned if that peak was not found. This produced a 217×728 matrix of peak heights as features, which was used in a lasso regression. We scored the test spectra for the same 728 peaks.

³Use of the value 0.005 means that peaks with positions less than 0.5% apart are considered the same peak, a fairly common assumption.

TABLE 18.2. *Results for the prostate data example. The standard deviation for the test errors is about 4.5.*

Method	Test Errors/108	Number of Sites
1. Nearest shrunk centroids	34	459
2. Lasso	22	113
3. Lasso on peaks	28	35

The prediction results for this application of the lasso to the peaks are shown in the last line of Table 18.2: it does fairly well, but not as well as the lasso on the raw spectra. However, the fitted model may be more useful to the biologist as it yields 35 peak positions for further study. On the other hand, the results suggest that there may be useful discriminatory information between the peaks of the spectra, and the positions of the lasso sites from line (2) of the table also deserve further examination.

18.4.2 The Fused Lasso for Functional Data

In the previous example, the features had a natural order, determined by the mass-to-charge ratio m/z . More generally, we may have functional features $x_i(t)$ that are ordered according to some index variable t . We have already discussed several approaches for exploiting such structure.

We can represent $x_i(t)$ by their coefficients in a basis of functions in t , such as splines, wavelets or Fourier bases, and then apply a regression using these coefficients as predictors. Equivalently, one can instead represent the coefficients of the original features in these bases. These approaches are described in Section 5.3.

In the classification setting, we discuss the analogous approach of penalized discriminant analysis in Section 12.6. This uses a penalty that explicitly controls the resulting smoothness of the coefficient vector.

The above methods tend to smooth the coefficients uniformly. Here we present a more adaptive strategy that modifies the lasso penalty to take into account the ordering of the features. The *fused lasso* (Tibshirani et al., 2005) solves

$$\min_{\beta \in \mathbb{R}^p} \left\{ \sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^{p-1} |\beta_{j+1} - \beta_j| \right\}. \quad (18.22)$$

This criterion is strictly convex in β , so a unique solution exists. The first penalty encourages the solution to be sparse, while the second encourages it to be smooth in the index j .

The difference penalty in (18.22) assumes an uniformly spaced index j . If instead the underlying index variable t has nonuniform values t_j , a natural generalization of (18.22) would be based on divided differences

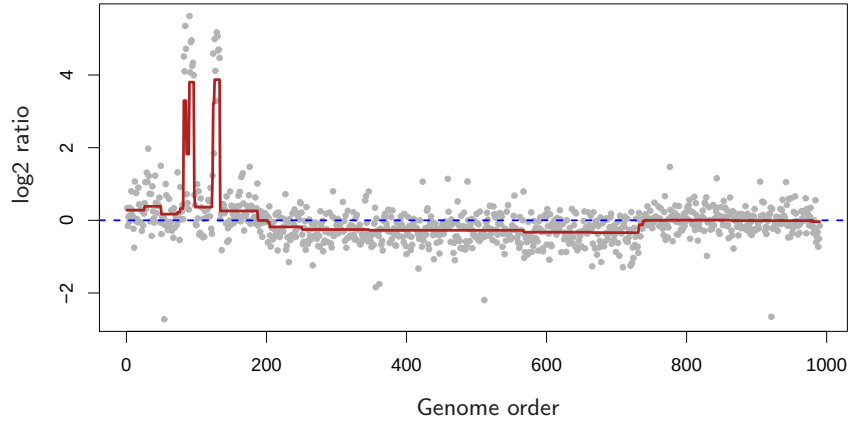


FIGURE 18.8. Fused lasso applied to CGH data. Each point represents the copy-number of a gene in a tumor sample, relative to that of a control (on the log base-2 scale).

$$\lambda_2 \sum_{j=1}^{p-1} \frac{|\beta_{j+1} - \beta_j|}{|t_{j+1} - t_j|}. \quad (18.23)$$

This amounts to having a penalty modifier for each of the terms in the series.

A particularly useful special case arises when the predictor matrix $\mathbf{X} = \mathbf{I}_N$, the $N \times N$ identity matrix. This is a special case of the fused lasso, used to approximate a sequence $\{y_i\}_1^N$. The *fused lasso signal approximator* solves

$$\min_{\beta \in \mathbb{R}^N} \left\{ \sum_{i=1}^N (y_i - \beta_0 - \beta_i)^2 + \lambda_1 \sum_{i=1}^N |\beta_i| + \lambda_2 \sum_{i=1}^{N-1} |\beta_{i+1} - \beta_i| \right\}. \quad (18.24)$$

Figure 18.8 shows an example taken from Tibshirani and Wang (2007). The data in the panel come from a Comparative Genomic Hybridization (CGH) array, measuring the approximate log (base-two) ratio of the number of copies of each gene in a tumor sample, as compared to a normal sample. The horizontal axis represents the chromosomal location of each gene. The idea is that in cancer cells, genes are often amplified (duplicated) or deleted, and it is of interest to detect these events. Furthermore, these events tend to occur in contiguous regions. The smoothed signal estimate from the fused lasso signal approximator is shown in dark red (with appropriately chosen values for λ_1 and λ_2). The significantly nonzero regions can be used to detect locations of gains and losses of genes in the tumor.

There is also a two-dimensional version of the fused lasso, in which the parameters are laid out in a grid of pixels, and a penalty is applied to the

first differences to the left, right, above and below the target pixel. This can be useful for denoising or classifying images. Friedman et al. (2007) develop fast generalized coordinate descent algorithms for the one- and two-dimensional fused lasso.

18.5 Classification When Features are Unavailable

In some applications the objects under study are more abstract in nature, and it is not obvious how to define a feature vector. As long as we can fill in an $N \times N$ *proximity* matrix of similarities between pairs of objects in our database, it turns out we can put to use many of the classifiers in our arsenal by interpreting the proximities as inner-products. Protein structures fall into this category, and we explore an example in Section 18.5.1 below.

In other applications, such as document classification, feature vectors are available but can be extremely high-dimensional. Here we may not wish to compute with such high-dimensional data, but rather store the inner-products between pairs of documents. Often these inner-products can be approximated by sampling techniques.

Pairwise distances serve a similar purpose, because they can be turned into centered inner-products. Proximity matrices are discussed in more detail in Chapter 14.

18.5.1 Example: String Kernels and Protein Classification

An important problem in computational biology is to classify proteins into functional and structural classes based on their sequence similarities. Protein molecules are strings of amino acids, differing in both length and composition. In the example we consider, the lengths vary between 75–160 amino-acid molecules, each of which can be one of 20 different types, labeled using letters. Here are two examples, of length 110 and 153, respectively:

IPTSALVKETLALLSTHRTLIIANETLRIPVPVHKHQLCTEEIFQGIGTLESQTVQGGTV
ERLFKNLSLIKYYIDGQKKKCGEERRRVNQFLDY**LQE**FLGVMNTEWI

PHRRDLCSRSIWLARKIRSDLTALTESYVKHQGLWSELTEAER**LQEN**LQAYRTFHVLLA
RLLEDQVVFHTPTTEGDFHQAIHTLLLQVAFAFYQIEELMILLEYKIPRNEADGMLFEKK
LWGLKVL**LQELS**QWTVRSIHDLRFISSHQTGIP

There have been many proposals for measuring the similarity between a pair of protein molecules. Here we focus on a measure based on the count of matching substrings (Leslie et al., 2004), such as the **LQE** above.

To construct our features, we count the number of times that a given sequence of length m occurs in our string, and we compute this number

for all possible sequences of length m . Formally, for a string x , we define a feature map

$$\Phi_m(x) = \{\phi_a(x)\}_{a \in \mathcal{A}_m} \quad (18.25)$$

where $\mathcal{A}_m$ is the set of subsequences of length m , and $\phi_a(x)$ is the number of times that “ a ” occurs in our string x . Using this, we define the inner product

$$K_m(x_1, x_2) = \langle \Phi_m(x_1), \Phi_m(x_2) \rangle, \quad (18.26)$$

which measures the similarity between the two strings x_1, x_2 . This can be used to drive, for example, a support vector classifier for classifying strings into different protein classes.

Now the number of possible sequences a is $|\mathcal{A}_m| = 20^m$, which can be very large for moderate m , and the vast majority of the subsequences do not match the strings in our training set. It turns out that we can compute the $N \times N$ inner-product matrix or *string kernel* $\mathbf{K}_m$ (18.26) efficiently using tree-structures, without actually computing the individual vectors. This methodology, and the data to follow, come from Leslie et al. (2004).⁴

The data consist of 1708 proteins in two classes—negative (1663) and positive (45). The two examples above, which we will call “ x_1 ” and “ x_2 ”, are from this set. We have marked the occurrences of subsequence **LQE**, which appears in both proteins. There are 20^3 possible subsequences, so $\Phi_3(x)$ will be a vector of length 8000. For this example $\phi_{LQE}(x_1) = 1$ and $\phi_{LQE}(x_2) = 2$.

Using software from Leslie et al. (2004), we computed the string kernel for $m = 4$, which was then used in a support vector classifier to find the maximal margin solution in this $20^4 = 160,000$ -dimensional feature space. We used 10-fold cross-validation to compute the SVM predictions on all of the training data. The orange curve in Figure 18.9 shows the cross-validated ROC curve for the support vector classifier, computed by varying the cut-point on the real-valued predictions from the cross-validated support vector classifier. The area under the curve is 0.84. Leslie et al. (2004) show that the string kernel method is competitive with, but perhaps not as accurate as, more specialized methods for protein string matching.

Many other classifiers can be computed using only the information in the kernel matrix; some details are given in the next section. The results for the nearest centroid classifier (green), and distance-weighted one-nearest neighbors (blue) are shown in Figure 18.9. Their performance is similar to that of the support vector classifier.

⁴We thank Christina Leslie for her help and for providing the data, which is available on our book website.

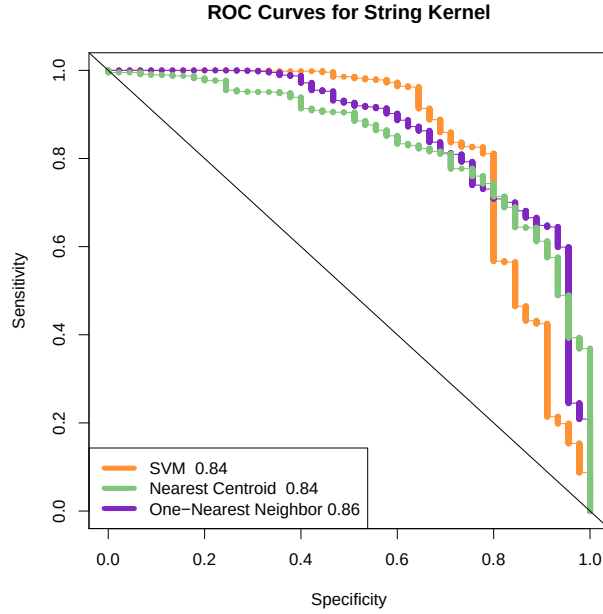


FIGURE 18.9. Cross-validated ROC curves for protein example using the string kernel. The numbers next to each method in the legend give the area under the curve, an overall measure of accuracy. The SVM achieves better sensitivities than the other two, which achieve better specificities.

18.5.2 Classification and Other Models Using Inner-Product Kernels and Pairwise Distances

There are a number of other classifiers, besides the support-vector machine, that can be implemented using only inner-product matrices. This also implies they can be “kernelized” like the SVM.

An obvious example is nearest-neighbor classification, since we can transform pairwise inner-products to pairwise distances:

$$\|x_i - x_{i'}\|^2 = \langle x_i, x_i \rangle + \langle x_{i'}, x_{i'} \rangle - 2\langle x_i, x_{i'} \rangle. \quad (18.27)$$

A variation of 1-NN classification is used in Figure 18.9, which produces a continuous discriminant score needed to construct a ROC curve. This distance-weighted 1-NN makes use of the distance of a test points to the closest member of each class; see Exercise 18.14.

Nearest-centroid classification follows easily as well. For training pairs (x_i, g_i) , $i = 1, \dots, N$, a test point x_0 , and class centroids $\bar{x}_k$, $k = 1, \dots, K$ we can write

$$\|x_0 - \bar{x}_k\|^2 = \langle x_0, x_0 \rangle - \frac{2}{N_k} \sum_{g_i=k} \langle x_0, x_i \rangle + \frac{1}{N_k^2} \sum_{g_i=k} \sum_{g_{i'}=k} \langle x_i, x_{i'} \rangle, \quad (18.28)$$